CHAPTER – 1
INTRODUCTION

The rapid advancement of digital communication technologies has transformed the way individuals and organizations exchange information. Messaging platforms have become fundamental tools for academic collaboration, professional coordination, and personal interaction. While these platforms provide convenience and speed, they also introduce significant cyber-security challenges. Among these challenges, the misuse of multimedia files—particularly images—for covert data transmission has become a growing concern.

Steganography is a technique used to conceal secret information within ordinary digital media such as images, audio files, or videos. Unlike cryptography, which secures data by converting it into an unreadable format, steganography hides the existence of the data itself. This makes detection considerably more difficult, as the file appears visually unchanged. Attackers exploit this property to embed malicious scripts, confidential information, or executable programming code inside image files and distribute them through communication systems.

Traditional chat applications primarily focus on encrypting messages during transmission and ensuring secure authentication. However, they generally do not perform deep inspection of multimedia content. As a result, an image containing hidden data can be transmitted and received without any analysis of its internal structure. If the concealed payload contains executable code or harmful instructions, it may compromise the receiving system or serve as a gateway for further cyber-attacks.

To address this emerging threat, there is a need for an intelligent communication system that not only enables real-time messaging but also actively monitors and analyzes multimedia content for hidden data. This project introduces a Chat Application that integrates a hybrid steganalysis framework combining deep learning and traditional algorithms to enhance security.

The proposed system employs SRNet (Spatial Rich Model Network), a deep convolutional neural network designed for steganalysis, to detect statistical irregularities in images that may indicate hidden content. SRNet is capable of identifying subtle pixel-level modifications introduced during steganographic embedding. However, detection alone is insufficient. Therefore, the system incorporates traditional extraction mechanisms, such as

Least Significant Bit (LSB) extraction, to retrieve concealed payloads from suspicious images.

Once the hidden data is extracted, the system reconstructs the binary payload into readable text format. A classification module then analyzes the reconstructed content to determine whether it contains programming code. The classifier uses pattern-based and structural analysis techniques to identify common syntactic characteristics of languages such as Python, C/C++, Java, JavaScript, HTML, and SQL. If executable or suspicious code is detected, the system automatically triggers a security alert, logs the event, and can optionally block or quarantine the image.

This hybrid architecture—combining deep learning-based detection (SRNet) with traditional extraction and rule-based classification—ensures higher reliability and practical usability. The deep learning component enhances detection accuracy, while the traditional modules provide interpretability and payload analysis. Together, they form a multi-layered defense mechanism within a real-time chat environment.

1.1 MOTIVATION

The increasing dependence on digital communication platforms has significantly improved global connectivity and information exchange. However, this rapid technological growth has also introduced new forms of cyber threats that exploit the weaknesses of existing communication systems. Among these threats, covert data transmission through steganography has emerged as a sophisticated and difficult-to-detect attack method.

Steganography enables malicious users to hide secret information inside seemingly harmless image files. These hidden payloads may contain confidential data, unauthorized instructions, or executable programming code. Since the visual appearance of the image remains unchanged, conventional security systems often fail to detect such concealed content. As a result, chat applications that allow unrestricted image sharing may unknowingly become channels for transmitting hidden malicious scripts.

Traditional security mechanisms primarily focus on encryption, authentication, and firewall protection. While these measures secure data during transmission, they do not analyze the internal structure of multimedia files for hidden information. This gap in security creates an opportunity for attackers to bypass detection systems by embedding harmful code within images using techniques such as Least Significant Bit (LSB) modification.

Recent advancements in deep learning have introduced powerful steganalysis models capable of detecting subtle statistical changes in images. SRNet (Spatial Rich Model

Network), a deep convolutional neural network designed specifically for steganography detection, has demonstrated high accuracy in identifying manipulated images. However, deep learning models alone cannot fully solve the problem, as detection must be followed by extraction and interpretation of the concealed payload.

This project is motivated by the need to develop a secure communication system that integrates both artificial intelligence and traditional analytical techniques. By combining SRNet-based detection with LSB extraction and rule-based code classification, the proposed hybrid model enhances both detection accuracy and practical applicability. The integration of these components ensures that the system not only identifies suspicious images but also reconstructs and analyzes hidden data to determine whether it contains executable programming code.

Another major motivation behind this project is the increasing concern regarding hidden code injection attacks. If programming code is secretly embedded within images and transmitted across systems, it may later be extracted and executed, leading to unauthorized access, data breaches, or system compromise. Therefore, incorporating automated code identification mechanisms within communication platforms is essential for proactive cyber defense.

Furthermore, the project aims to bridge the gap between theoretical steganalysis research and real-world application. While many studies focus on developing detection algorithms in isolation, fewer systems integrate these techniques into practical, real-time communication environments. This project demonstrates how steganalysis, payload extraction, and code classification can be embedded directly into a functional chat application.

In summary, the motivation for this project arises from:

- The growing misuse of steganography for malicious purposes.
- The lack of hidden-content analysis in conventional chat systems.
- The need for intelligent, AI-driven cyber-security solutions.

1.2 OBJECTIVE

The primary objective of this project is to design and develop an intelligent Chat Application that not only enables real-time communication but also incorporates advanced cyber-security mechanisms to detect and analyze hidden data embedded within image files. The system aims to enhance communication security by integrating deep learning–based steganalysis with traditional extraction and classification techniques.

The specific objectives of the project are as follows:

1. To Develop a Real-Time Communication System

To design and implement a Chat Application that supports the exchange of text messages and images between two clients through a centralized backend server. The system should ensure smooth, reliable, and real-time communication.

2. To Integrate Deep Learning-Based Steganography Detection

To incorporate a steganalysis model, specifically SRNet (Spatial Rich Model Network), capable of detecting subtle pixel-level modifications in images. The objective is to automatically analyze every received image and determine whether it potentially contains hidden data.

3. To Implement Hidden Data Extraction Mechanism

To develop a traditional extraction module using techniques such as Least Significant Bit (LSB) decoding to retrieve concealed payloads from images identified as suspicious. This ensures that detection is followed by actual data recovery for further analysis.

4. To Reconstruct and Process Extracted Payload

To convert the extracted binary data into readable text format. The system should handle payload reconstruction accurately to avoid data corruption or misinterpretation.

5. To Identify Embedded Programming Code

To design a classification module that analyzes the reconstructed payload and determines whether it contains programming code. The classifier should identify common syntactic patterns associated with languages such as Python, C/C++, Java, JavaScript, HTML, and SQL.

6. To Generate Automated Security Alerts

To implement a security response mechanism that alerts the user if hidden data or executable code is detected. The system should also provide options to block, quarantine, or discard suspicious images.

7. To Maintain Detection and Security Logs

To record all detection results, extracted payload summaries, and security actions in a structured log file for auditing and analysis purposes.

8. To Design a Hybrid Security Architecture

To combine artificial intelligence–based detection (SRNet) with traditional extraction and rule-based classification algorithms, creating a multi-layered security framework that enhances detection accuracy and interpretability.

1.3 PROBLEM STATEMENT

Modern chat systems allow users to exchange images without verifying whether those images contain hidden or malicious content. Steganography techniques enable attackers to embed secret data, including executable programming code, within image files without visibly altering them. Since conventional communication platforms do not analyze the internal structure of images, such concealed payloads can be transmitted undetected.

The problem addressed in this project is to design a Chat Application capable of automatically detecting steganographic content in received images using a hybrid approach that combines deep learning–based detection (SRNet) with traditional extraction and classification techniques. The system must identify hidden data, extract the concealed payload, determine whether it contains programming code, and initiate appropriate security actions to prevent potential cyber threats.

CHAPTER – 2

LITERATURE SURVEY

The rapid advancement of steganographic techniques, particularly in the spatial domain, has significantly increased the need for intelligent and robust steganalysis systems. Traditional statistical detection approaches have gradually been replaced by deep learning-based frameworks capable of identifying subtle embedding artifacts introduced by techniques such as Least Significant Bit (LSB) substitution. This section reviews important research contributions in chronological order, beginning with the most recent work.

1. Deep Learning-Based Multi-Class Detection of LSB Steganography in Digital Images – 2025

This study, published in November 2025, introduced a deep learning framework for both binary detection and multi-class classification of LSB steganographic payloads.

The proposed system distinguished between clean images and stego images containing different embedded payloads such as HTML, JavaScript, PowerShell scripts, URLs, and Ethereum-related data.

The authors evaluated multiple architectures including a custom Convolutional Neural Network (CNN), hybrid CNN-LSTM, CNN-GRU models, and Vision Transformer (ViT). A major finding of the study was that image resizing significantly degrades detection performance because LSB artifacts are highly sensitive to resolution changes. Among all evaluated models, the CNN-GRU demonstrated superior generalization capability, especially on external datasets, due to its ability to combine spatial feature extraction with sequential dependency modelling.

2. Image Steganalysis Using Active Learning and Hyperparameter Optimization – 2025

This research proposed a steganalysis framework that integrates active learning with hyperparameter optimization techniques to improve detection efficiency. The study aimed to reduce dependency on large labeled datasets while maintaining high detection accuracy. By intelligently selecting informative training samples, the framework improved scalability and computational efficiency in deep learning-based steganalysis systems.

3. Deep Learning for Steganalysis: Evaluating Model Robustness Against Image Transformations – 2025

This study analyzed the robustness of deep learning-based steganalysis models under common image transformations such as resizing, compression, and noise addition. The results demonstrated that many CNN-based models suffer performance degradation when preprocessing operations alter the image structure.

The research highlighted the importance of preserving original image fidelity for effective LSB detection and identified robustness as a critical area for future improvement.

4. Deep Learning-Based Steganalysis for Detection and Classification of Hidden Content – 2024

This study proposed a deep learning framework capable of detecting and classifying hidden content embedded in digital images. The authors emphasized hierarchical feature extraction and domain-specific preprocessing techniques to enhance detection sensitivity. Their results demonstrated improved accuracy compared to traditional machine learning approaches.

5. A Convolutional Neural Network to Detect Possible Hidden Data in Spatial Domain Images – 2023

This research introduced a CNN-based steganalysis model incorporating high-pass filtering mechanisms to amplify embedding noise. The approach improved detection accuracy, particularly at low embedding rates. The study confirmed that convolutional architectures can effectively learn statistical irregularities introduced by LSB embedding without relying on handcrafted features.

6. STEG-XAI: Explainable Steganalysis in Images Using Neural Networks – 2023

This work focused on improving the interpretability of deep learning-based steganalysis models. By integrating explainable AI techniques, the authors identified image regions contributing to detection decisions. This approach addressed the black-box limitation of neural networks and enhanced trust in security-sensitive applications.

7. Detection of Steganographic Threats Targeting Digital Images in Heterogeneous Ecosystems Through Machine Learning – 2022

This study investigated machine learning-based detection of steganographic threats in real-world digital environments. The research highlighted the increasing use of LSB techniques for embedding malicious payloads and emphasized the need for intelligent detection systems capable of handling diverse image types.

8. Image Steganography by Deep CNN Auto-Encoder Networks – 2020

This study proposed a deep convolutional autoencoder framework for image steganography and extraction. The model consisted of a hiding network and a revealing network trained end-to-end to embed and recover secret images. The research demonstrated that deep CNN architectures can effectively learn embedding and extraction mappings without relying solely on traditional LSB substitution techniques. The work significantly contributed to the transition from manual embedding strategies to deep learning-driven steganographic systems.

9. Deep Learning Hierarchical Representations for Image Steganalysis – 2017

This research introduced a deep convolutional neural network designed specifically for steganalysis using hierarchical feature representations. The model incorporated multiple convolutional layers to capture embedding noise patterns introduced by spatial-domain steganography. The study demonstrated improved detection accuracy compared to traditional machine learning approaches based on handcrafted statistical features.

10. Structural Design of Convolutional Neural Networks for Steganalysis – 2016

This foundational work focused on optimizing CNN architecture for steganalysis tasks. The authors emphasized the importance of specialized filter initialization, high-pass preprocessing, and network depth in detecting subtle pixel-level embedding artifacts. The study established convolutional neural networks as a superior alternative to conventional feature-based steganalysis methods and laid the groundwork for many subsequent deep learning-based detection models.

CHAPTER – 3
SYSTEM ANALYSIS

System analysis is a crucial phase in the software development lifecycle that focuses on understanding the existing environment, identifying system limitations, and defining the functional and technical requirements of the proposed solution. It provides a structured evaluation of current technologies and highlights the necessity for improvement.

In this project, system analysis concentrates on examining conventional chat applications and identifying their security weaknesses, particularly in handling multimedia content. With the increasing use of steganography for concealing sensitive or malicious data inside images, there is a growing need for communication systems that go beyond basic encryption and incorporate intelligent detection mechanisms. The objective of this analysis is to justify the need for an enhanced Chat Application that integrates hybrid steganography detection and hidden code identification capabilities.

3.1 EXISTING SYSTEM

Existing chat applications are primarily designed to enable real-time communication between users through text messaging, image sharing, and file transfer functionalities. These systems typically operate using client-server architecture, where messages and multimedia files are transmitted through a centralized backend server.

Most modern chat platforms implement security mechanisms such as user authentication, session management, and encryption protocols to protect data during transmission. Encryption ensures confidentiality by preventing unauthorized access while data is being transferred across networks.

Image sharing is a standard feature in these applications, allowing users to send and receive multimedia content instantly. Upon receiving an image, the system usually verifies file type and size before storing or displaying it to the recipient. The image is then rendered directly to the user interface without performing deep content inspection at the pixel level.

Overall, existing systems focus on providing seamless communication, efficient data transfer, and secure transmission channels. However, their core design emphasizes functionality and transmission security rather than advanced multimedia content analysis.

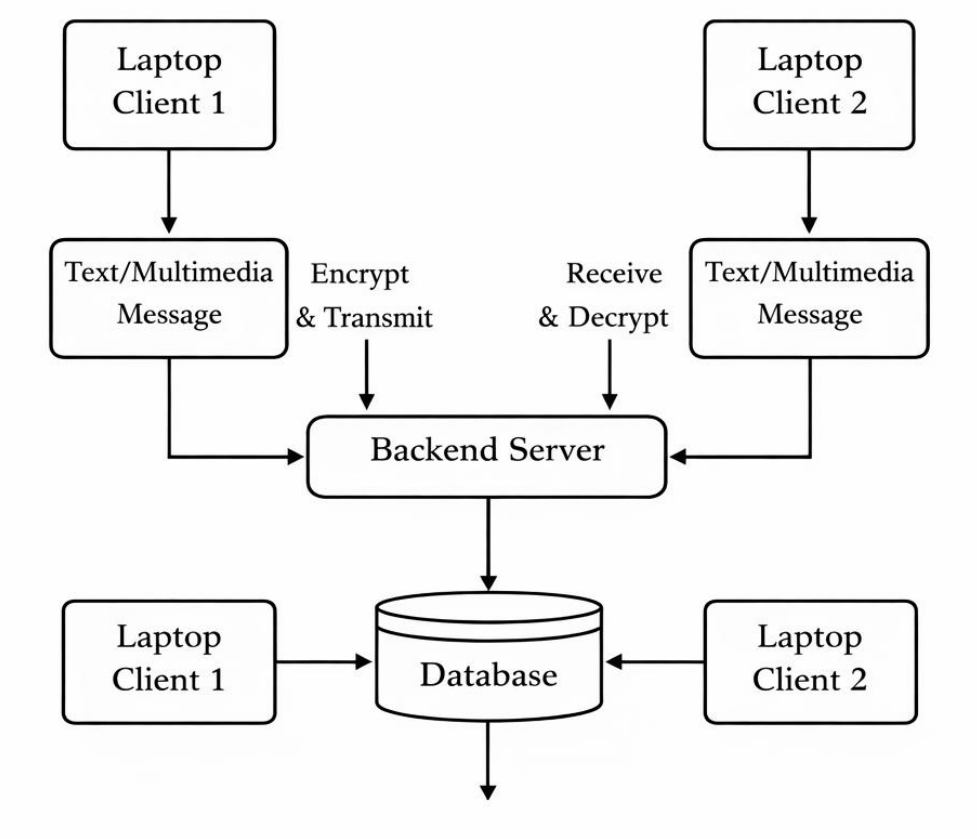

Fig 3.1.1 Existing System

3.1.1 DEMERITS

- **No Steganography Detection:** Existing chat systems do not analyze images for hidden data embedded using techniques such as LSB substitution.
- **No Payload Extraction:** Even if an image contains concealed information, there is no mechanism to extract or reconstruct the hidden content.
- **No Code Identification:** The system cannot determine whether embedded text contains executable or malicious programming code.
- **Surface-Level Security:** Security measures focus only on encryption during transmission and not on deep content inspection.
- **No Pixel-Level Analysis:** Images are displayed directly without analyzing internal pixel modifications.
- **No Automated Alerts:** Users are not notified when suspicious or manipulated images are received.
- **Lack of Threat Classification:** Hidden content, if present, is not categorized or analyzed for potential risks.
- **No Security Logging:** There is no structured logging mechanism to record suspicious multimedia activities.

- **Vulnerability to Covert Attacks:** Steganography-based hidden data can pass through the system undetected.
- **Limited Cybersecurity Intelligence:** Existing systems prioritize communication efficiency over proactive multimedia threat detection.

3.2 PROPOSED SYSTEM

The proposed system is an advanced Chat Application designed to enhance communication security by integrating a hybrid steganography detection framework. Unlike conventional chat platforms that focus only on secure data transmission, the proposed system incorporates intelligent multimedia content analysis to detect hidden information embedded within images.

The system follows a client-server architecture, where two users communicate through a centralized backend server. In addition to supporting real-time text messaging and image sharing, the system automatically analyzes every received image before it is displayed to the user.

The core strength of the proposed system lies in its hybrid detection approach, which combines deep learning and traditional algorithms. Initially, a deep learning-based steganalysis model (SRNet) examines the image to detect subtle pixel-level modifications that may indicate hidden data. If the image is flagged as suspicious, a Least Significant Bit (LSB) extraction module retrieves the concealed payload.

Once the hidden data is extracted, it is reconstructed into readable text format. A classification module then analyzes the extracted content to determine whether it contains programming code such as Python, C/C++, Java, JavaScript, HTML, or SQL. If executable or suspicious code is detected, the system generates an alert and logs the event for security monitoring.

The proposed system therefore provides a multi-layered security mechanism that ensures not only secure transmission but also proactive detection and analysis of covert threats embedded within multimedia files. This approach significantly enhances cybersecurity within digital communication environments.

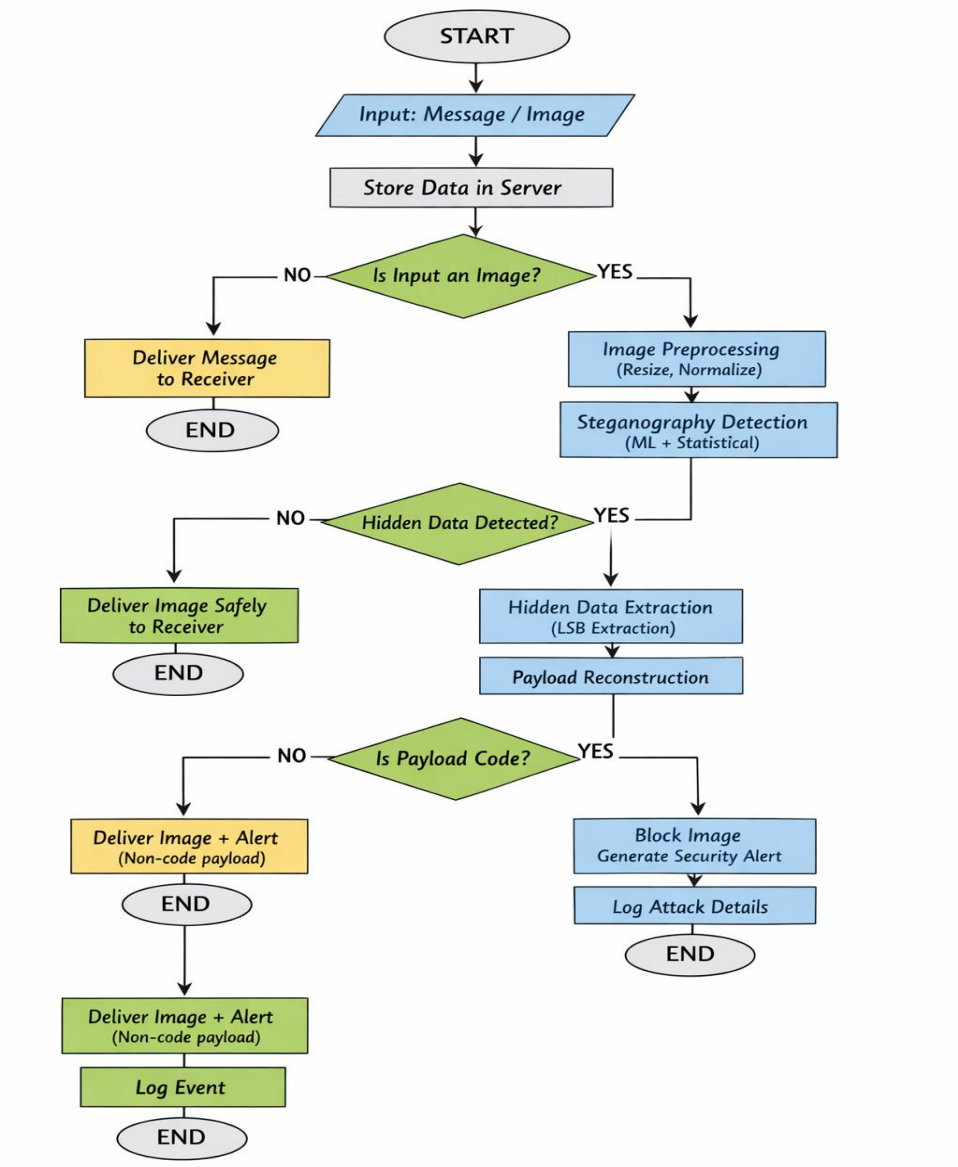

Fig 3.2.1 Proposed System

3.2.1 MERITS OF PROPOSED SYSTEM

The proposed Chat Application introduces several significant improvements over existing systems by integrating intelligent multimedia content analysis with real-time communication. The major merits of the proposed system are as follows:

• **Automatic Steganography Detection:** The system automatically analyzes received images using a deep learning-based steganalysis model to detect hidden data.

• **Hybrid Detection Framework:** Combines SRNet-based detection with traditional LSB extraction techniques for improved reliability and accuracy.

• **Hidden Payload Extraction:** Capable of extracting concealed data embedded within image files.

• **Programming Code Identification:** Analyzes extracted content to determine whether it contains executable programming code.

• **Multi-Layer Security Mechanism:** Implements detection, extraction, reconstruction, and classification stages to enhance threat analysis.

• **Real-Time Threat Monitoring:** Automatically scans images before displaying them to the user.

• **Security Alert System:** Generates alerts when suspicious or malicious content is detected.

• **Event Logging Mechanism:** Maintains structured logs for monitoring and auditing security-related activities.

• **Improved Cybersecurity Awareness:** Protects users from covert data transmission and hidden code injection attacks.

• **Scalable and Modular Design:** The system architecture allows future enhancements such as multi-user support and advanced AI-based models.

3.3 FEASIBILITY STUDY

The proposed Chat Application is technically feasible as it can be developed using standard and widely supported technologies such as FastAPI for the backend and Python for client-side implementation. Deep learning frameworks like PyTorch or TensorFlow enable the integration of the SRNet model for steganography detection, while image processing libraries support LSB-based payload extraction and classification. The system does not require specialized hardware and can operate efficiently on standard computing devices used in academic environments.

From an economic perspective, the system is cost-effective since it relies primarily on open-source tools, frameworks, and libraries. No additional licensing fees or expensive infrastructure investments are required. The development and deployment can be carried out using existing institutional resources, making the overall implementation financially viable.

Operationally, the proposed system is user-friendly and designed to function seamlessly within a real-time communication environment. Image scanning, detection, extraction, and classification processes are performed automatically in the background without interrupting normal chat activities. The alert and logging mechanisms further enhance usability by providing timely notifications while maintaining smooth communication flow.

3.3.1 ECONOMIC FEASIBILITY

The proposed Chat Application is economically feasible as it is developed using open-source technologies and freely available development frameworks. Tools such as FastAPI, Python, and deep learning libraries like PyTorch or TensorFlow eliminate the need for costly software licenses. Additionally, the system does not require specialized hardware infrastructure, as it can operate efficiently on standard laptops and computing devices commonly available in academic institutions.

The implementation cost is minimal because the development process primarily involves software configuration, model integration, and system testing using existing resources. Since the project is designed within an academic environment, infrastructure expenses are limited, and no additional financial investment is required for deployment. Therefore, the proposed system is financially practical and sustainable.

3.3.2 TECHNICAL FEASIBILITY

The proposed Chat Application is technically feasible as it is built using well-established and widely supported technologies. The backend can be implemented using FastAPI, which provides efficient API handling and real-time communication support, while the client-side application can be developed using Python. The integration of deep learning models such as SRNet for steganography detection is supported by frameworks like PyTorch or TensorFlow. Additionally, image processing libraries enable efficient implementation of LSB-based payload extraction and reconstruction mechanisms.

The system architecture is modular, allowing seamless integration of detection, extraction, and classification components without affecting core communication functionality. The hardware requirements are moderate, and the system can operate on standard computing devices without the need for high-end servers or specialized infrastructure. Therefore, the technical resources, tools, and expertise required for development and deployment are readily available, making the project technically practical and achievable.

3.3.3 SOCIAL FEASIBILITY

The proposed Chat Application is socially feasible as it enhances user security without disrupting normal communication activities. The system operates in the background, automatically analyzing received images for hidden data while maintaining a smooth and user-friendly interface. Since users are not required to perform additional manual steps for security verification, the application remains convenient and accessible to individuals with basic technical knowledge.

Furthermore, the integration of automated steganography detection promotes cybersecurity awareness and responsible digital communication. By alerting users to potentially suspicious multimedia content, the system helps prevent covert data transmission and hidden code-based attacks. This contributes positively to digital safety practices in academic, professional, and personal communication environments, making the system socially acceptable and beneficial.

3.4 SYSTEM SPECIFICATIONS

The proposed Chat Application is developed using Python as the primary programming language, with FastAPI serving as the backend framework for handling API requests and real-time communication. The system utilizes deep learning frameworks such as PyTorch or TensorFlow for implementing the SRNet-based steganography detection model, along with image processing libraries for LSB payload extraction and reconstruction. The application operates on a client-server architecture and requires a standard computing device with a minimum of 8 GB RAM, Intel i5 processor or equivalent, and adequate storage for model files and chat data. The system is compatible with modern operating systems such as Windows, Linux, or macOS and supports internet connectivity for real-time message and image transmission.

3.4.1 FUNCTIONAL REQUIREMENTS

The proposed Chat Application is designed to perform specific functions that ensure secure real-time communication along with automated steganography detection and hidden code analysis. The primary functional requirements of the system are listed below:

1. **User Registration and Authentication:** The system must allow users to register and log in securely. The system shall validate user credentials before granting access to chat functionalities.
2. **Real-Time Messaging:** The system shall allow users to send and receive text messages in real time through a centralized backend server. Messages shall be stored in a database for retrieval when required.
3. **Image Uploading and Transmission:** Users must be able to upload and send image files through the chat interface. The system shall verify file type and size before processing.
4. **Automatic Image Scanning:** The system shall automatically scan every received image before displaying it to the user. The scanning process shall run in the background without affecting communication flow.

5. **Steganography Detection Module:** The system shall analyze images using a deep learning-based steganalysis model (e.g., SRNet) to determine whether hidden data is present.
6. **Hidden Data Extraction Module:** If an image is identified as suspicious, the system shall apply an LSB-based extraction algorithm to retrieve the concealed payload.
7. **Payload Reconstruction:** The system shall convert extracted binary data into readable text format for further analysis.
8. **Programming Code Classification:** The system shall analyze reconstructed text to identify whether it contains programming code such as Python, C/C++, Java, JavaScript, HTML, or SQL. The system shall generate a confidence indication for detected code patterns.
9. **Security Alert and Logging:** The system shall notify users if suspicious or malicious hidden content is detected. The system shall maintain logs of detection results, extracted content summaries, and security actions.
10. **Model Training and Updating:** The deep learning detection model shall support training and retraining with updated datasets to improve detection accuracy over time. The system shall allow model updates without affecting core communication functionality.

3.4.2 NON-FUNCTIONAL REQUIREMENTS

1. **Security Requirements:**

   • The system must ensure secure transmission of messages and images using encryption protocols.

   • Sensitive user data and chat history must be protected at rest and during transmission.

   • Secure authentication and session management mechanisms must be implemented.

   • The system must prevent unauthorized access to stored images and detection logs.
2. **Performance Requirements:**

   • The system should support real-time messaging without noticeable delay.

   • Image scanning and steganography detection must complete within an acceptable response time.

   • Model inference time for steganalysis should not exceed a predefined threshold to maintain smooth communication.

   • The system should handle multiple message and image transfers efficiently.

3. **Accuracy and Reliability:**

   • The steganography detection model should maintain high detection accuracy across different image types.

   • The system should minimize false positives and false negatives in hidden code detection.

   • Payload extraction and reconstruction must be reliable and consistent.

   • The system should operate continuously without unexpected crashes or failures.

4. **Usability Requirements:**

   • The application should provide a simple and intuitive user interface.

   • Clear notifications must be displayed when suspicious content is detected.

   • Error messages should be informative and easy to understand.

   • The system should operate automatically without requiring technical expertise from users.

3.4.3 SOFTWARE AND HARDWARE REQUIREMENTS

System requirements specify the hardware and software components necessary for the proper functioning of the Chat Application integrated with hybrid steganography detection.

HARDWARE SPECIFICATIONS:

- Processor : Intel i5 Processor or equivalent
- RAM : 8 GB (minimum recommended)
- Hard Disk : 256 GB (minimum)
- Internet Connection : Required for real-time communication
- GPU (Optional) : For faster deep learning model training and inference

SOFTWARE SPECIFICATIONS:

- Operating System : Windows 10/11, Linux, or macOS
- Server-Side Framework : FastAPI (Python 3.10 or above)
- Client-Side Application : Python-based interface
- Database : SQLite / PostgreSQL
- Communication Protocol : HTTP / WebSockets

Libraries Used:

- PyTorch / TensorFlow (for SRNet steganalysis model)
- OpenCV (for image processing and LSB extraction)
- NumPy (for numerical operations)
- Pillow (PIL) (for image handling)

- Scikit-learn (for classification utilities)
- Uvicorn (ASGI server for FastAPI)
- SQLAlchemy (for database interaction)

Development Tools:

- IDE : Visual Studio Code
- API Testing Tool : Postman
- AI Development Platform : Antigravity (for model integration, debugging, and optimization support)
- Version Control : Git & GitHub
- Deployment Platforms : Render (Backend Deployment), Vercel (Frontend Deployment)
- ASGI Server : Uvicorn

CHAPTER – 4

SYSTEM DESIGN

SYSTEM DESIGN

It is a model that provides the project detailed structure the figure below represents our system architecture for the project.

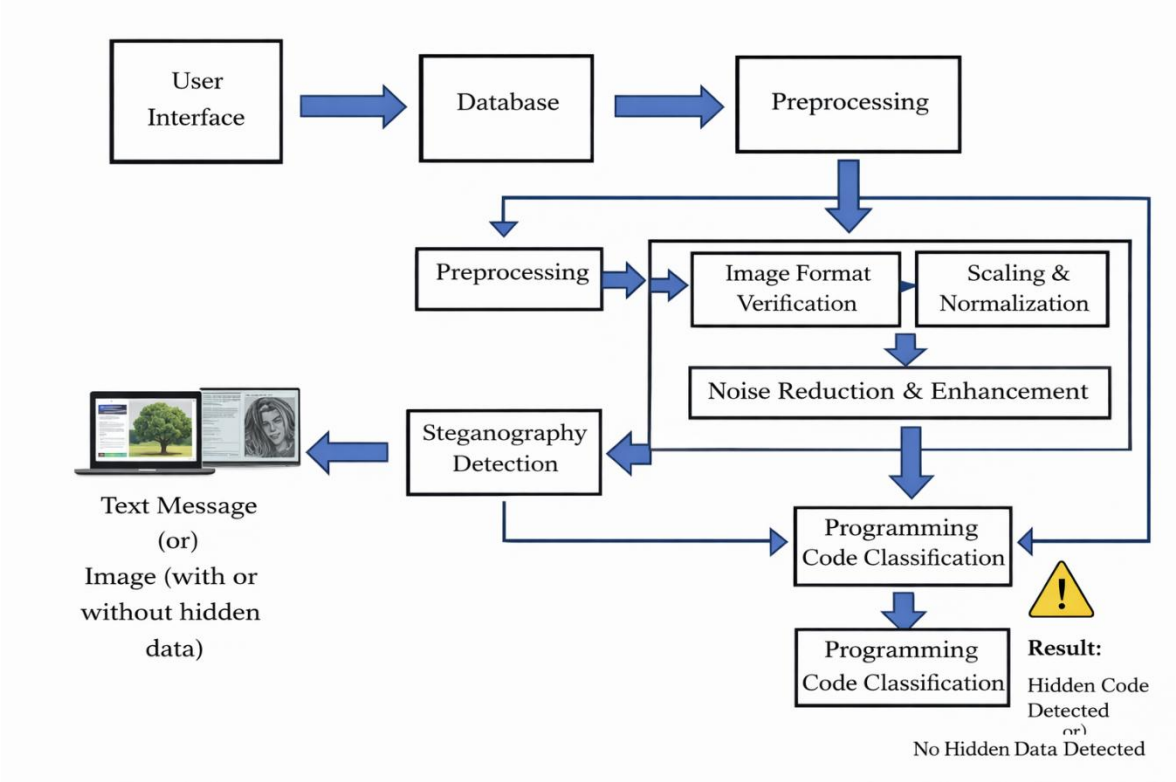

Fig 4.1.1 System Design

1.Image Input Module:

This module is responsible for receiving user-shared images through the Chat Application interface. It accepts images sent during chat communication and verifies the file format and size before processing. The system supports common formats such as PNG and JPEG. After validation, the image is forwarded to the analysis pipeline for further inspection. Text messages are transmitted normally, while image files are routed to the detection module automatically.

2.Image Analysis and Preprocessing Module:

Before steganography detection, the image undergoes several preprocessing steps to prepare it for analysis. These steps include image format verification, resolution checking, resizing to a fixed dimension suitable for the detection model, normalization of pixel values,

and optional noise filtering. These procedures ensure consistency in model input and help preserve subtle pixel-level modifications introduced by steganographic embedding.

3.Steganography Detection Module (SRNet):

This module forms the core of the system's security mechanism. A deep learning-based steganalysis model (e.g., SRNet) analyzes the image to detect hidden data embedded using spatial-domain techniques such as LSB substitution. The model identifies minute statistical irregularities and embedding artifacts within pixel values. Based on inference results, the image is classified as either clean or suspicious.

4.Hidden Data Extraction Module:

If the image is identified as suspicious, the system activates the LSB payload extraction mechanism. This module retrieves concealed binary data from the least significant bits of image pixels. The extracted binary stream is then converted into readable text through payload reconstruction procedures.

5.Programming Code Classification Module:

After reconstruction, the extracted text is analyzed to determine whether it contains programming code. The system checks for syntactic patterns, keywords, and structural characteristics of languages such as Python, C/C++, Java, JavaScript, HTML, or SQL. The module provides a classification result along with a confidence indication.

6.Output and Security Alert Module:

The system displays the final result to the user, indicating whether hidden data or programming code has been detected. If suspicious content is found, the application generates a warning notification and logs the event for security monitoring. If no hidden data is detected, the image is safely displayed within the chat interface.

7.User Interface Module:

The Chat Application provides an intuitive interface that allows users to send text messages and images seamlessly. The detection process runs automatically in the background without interrupting normal communication. Users receive clear notifications regarding security status while maintaining a smooth chat experience.

4.1 MODULES AND THEIR DESCRIPTION

The proposed Chat Application consists of multiple interconnected modules that work together to provide secure communication and automated steganography detection. Unlike traditional systems that rely on a single classification model, this project implements a hybrid architecture combining deep learning–based steganalysis with traditional extraction and code classification mechanisms. The major modules of the system are described below.

1. Steganography Detection Module (SRNet)

The steganography detection module is the core deep learning component of the system. It uses a Spatial Rich Model Network (SRNet), a convolutional neural network architecture specifically designed for steganalysis. The model analyzes pixel-level statistical variations introduced by LSB-based embedding techniques.

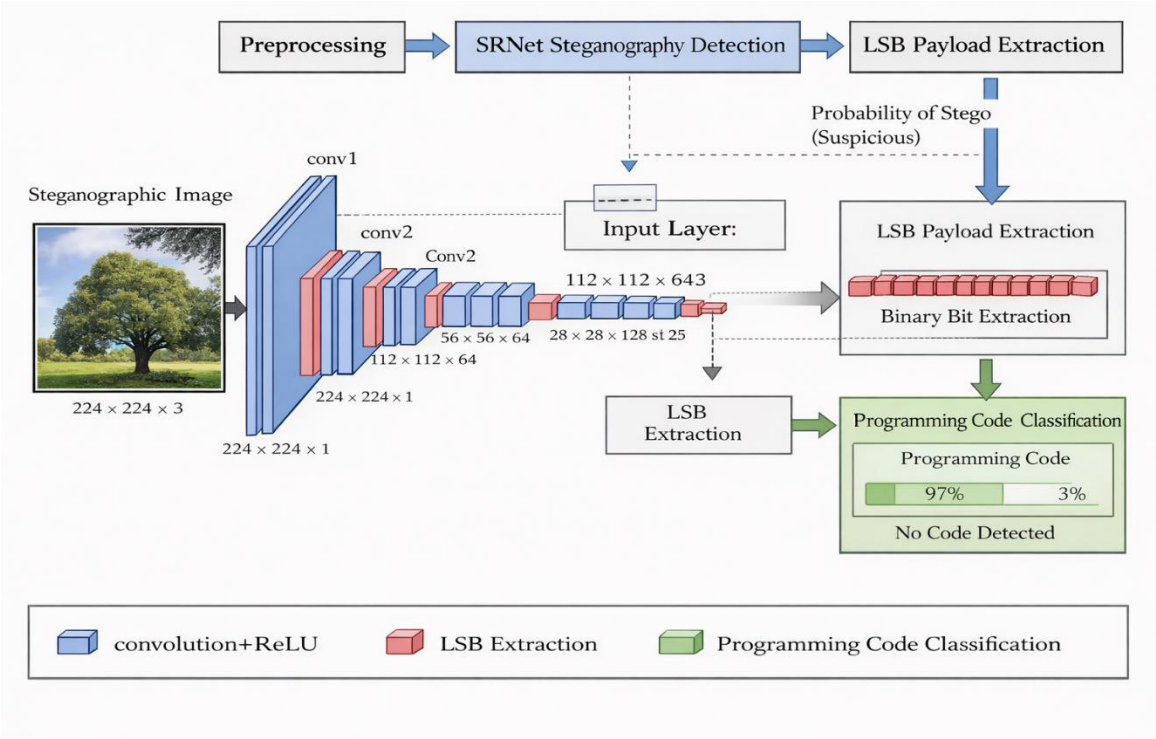

Fig 4.1.2 SRNet Model

The architecture consists of:

- **Input Layer:** Accepts RGB images resized to a fixed dimension suitable for the model.
- **Convolutional Layers:** Extract spatial features and embedding artifacts from image pixels.
- **Batch Normalization & Activation (ReLU):** Improve training stability and introduce non-linearity.
- **Residual Connections:** Enhance deep feature learning and improve gradient flow.
- **Global Pooling Layer:** Reduces feature maps into compact representations.
- **Output Layer:** Produces a probability score indicating whether the image is clean or contains hidden data.

This module performs binary classification (Stego / Non-Stego).

2. Hidden Data Extraction Module (LSB Extractor)

If the detection module classifies an image as suspicious, the system activates the extraction module. This module applies a Least Significant Bit (LSB) extraction algorithm to retrieve concealed binary data embedded within pixel values.

The extraction process includes:

- Reading pixel values
- Extracting least significant bits
- Reconstructing binary sequence
- Converting binary data into readable text

This module bridges the gap between detection and content analysis.

3. Payload Reconstruction Module

The extracted binary stream is processed to reconstruct meaningful textual data. The system removes padding bits, identifies termination markers, and converts the binary sequence into ASCII or UTF-8 text format. This ensures that the extracted payload is correctly interpreted before classification.

4. Programming Code Classification Module

This module analyzes the reconstructed text to determine whether it contains executable programming code. The classifier uses:

- Keyword-based pattern matching
- Structural syntax analysis
- Regular expressions for language-specific constructs

It checks for programming languages such as:

- Python
- C / C++
- Java
- JavaScript
- HTML
- SQL

The module outputs a classification result along with a confidence indication.

5. Security Alert and Logging Module

If hidden programming code is detected, this module generates a warning notification for the user. It also logs:

- Image metadata

- Detection result
- Extracted payload summary
- Timestamp

This ensures traceability and improves cybersecurity monitoring.

4.2 MODULES

4.2.1 INPUT DESIGN

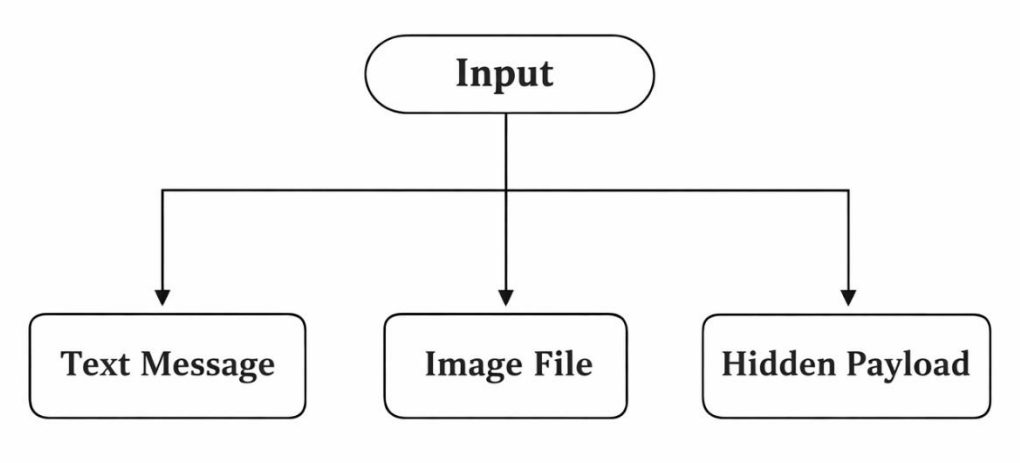

Fig: 4.2.1 INPUT DESIGN

Input design refers to how data is entered into the system and how the system accepts it.

1. **User Credentials**
    - Username
    - Password
    - Used for authentication and login
2. **Text Messages**
    - Users type and send messages in real time
3. **Image Files**
    - Users upload images (PNG / JPEG)
    - Images may contain hidden data
    - System validates file type and size
4. **Model Input**
    - Uploaded image is resized and normalized
    - Image is passed to SRNet for steganography detection

4.2.2 OUTPUT DESIGN

Output design refers to how the system displays results after processing.

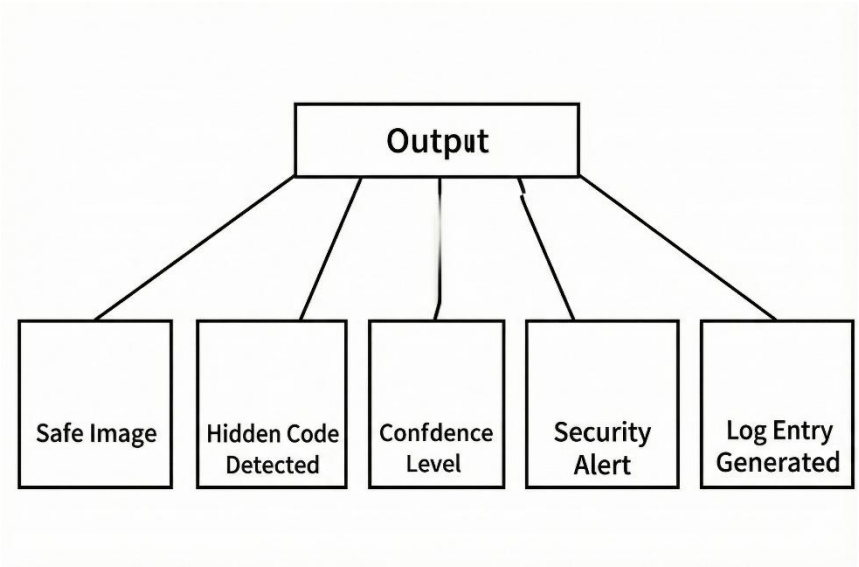

Fig: 4.2.2 OUTPUT DESIGN

1. **Delivered Text Message**
   - Message shown to receiver
2. **Image Display**
   - Clean image displayed normally
3. **Detection Result**
   - "No Hidden Data Detected"
   - OR
   - "Hidden Data Detected"
4. **Code Classification Result**
   - "Programming Code Detected"
   - OR
   - "No Executable Code Found"
5. **Security Alert**
   - Warning notification if suspicious content found
6. **Log Entry**
   - Detection result stored in system logs

4.3 UML DIAGRAMS

UML diagrams are a standardised way to visualise the design of a software system .they helped developers and stack holders understand ,design and document software architecture and components.

4.3.1 USE CASE

A Use Case UML diagram is created to represent the working process of the Chat Application. The primary actor is the User, who interacts with the System to perform actions such as registration, login, sending messages, and uploading images. When an image is uploaded, the system automatically performs steganography detection, hidden data extraction, and programming code classification. The oval-shaped elements in the diagram represent these individual functionalities. The diagram clearly illustrates the interaction flow between the user and the automated security processes of the system.

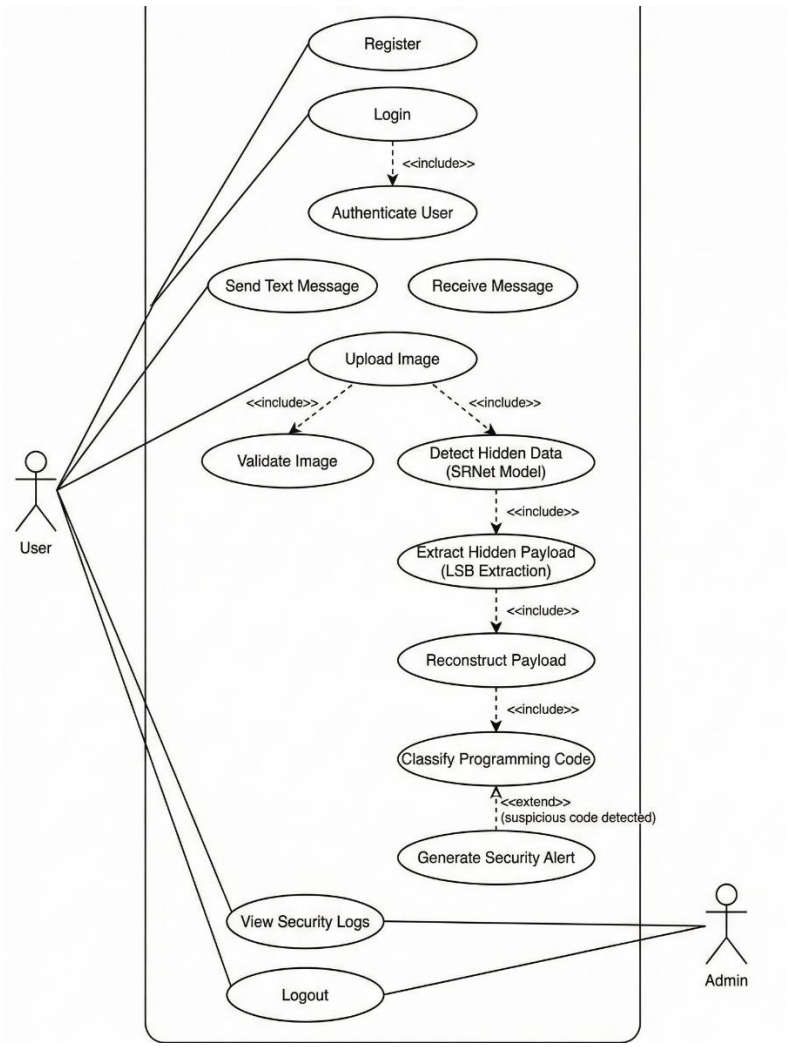

Fig: 4.3.1 USE CASE

4.3.2 ACTIVITY DIAGRAM

Here we have represented out project work in the form of activity diagram, here is clearly represent that there will be a starting point and ending point where from starting point we can clearly see that the image is uploaded and at the ending point the outcome of the uploaded image is represented.

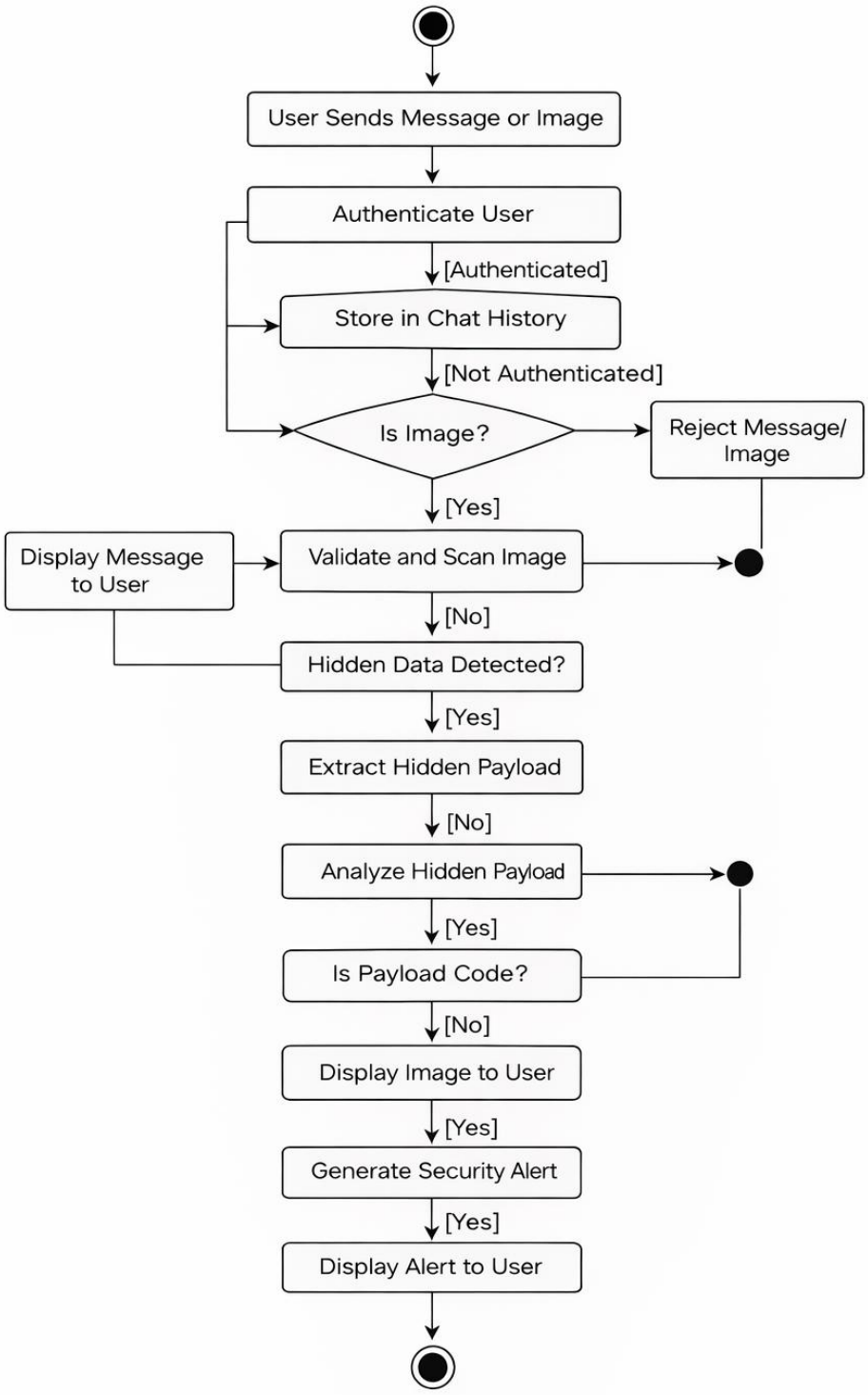

Fig: 4.3.2 ACTIVITY DIAGRAM

4.3.3 DEPLOYMENT DIAGRAM

This is deployment diagram where it is easy and simple to represent the working process and easy to understand by the user and other systems, whereas here we represented three blocks one is system and other is user and dataset.

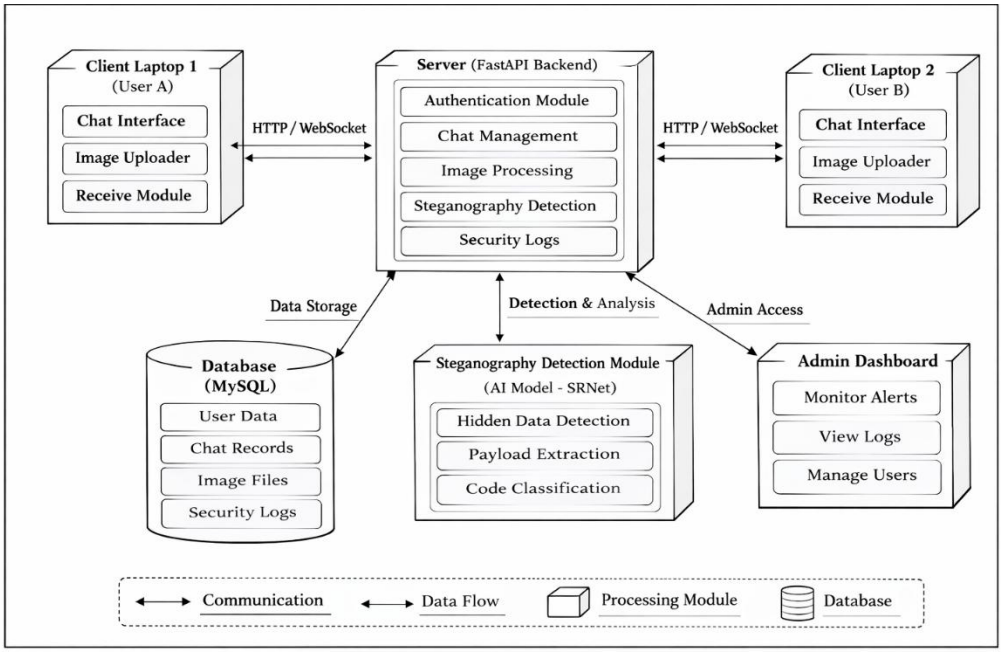

Fig: 4.3.3 DEPLOYMENT DIAGRAM

4.3.4 CLASS DIAGRAM

This is a class diagram system which is represented in three blocks one is system and another is user and dataset where there are all system working, another is user where it mention the outcomes.

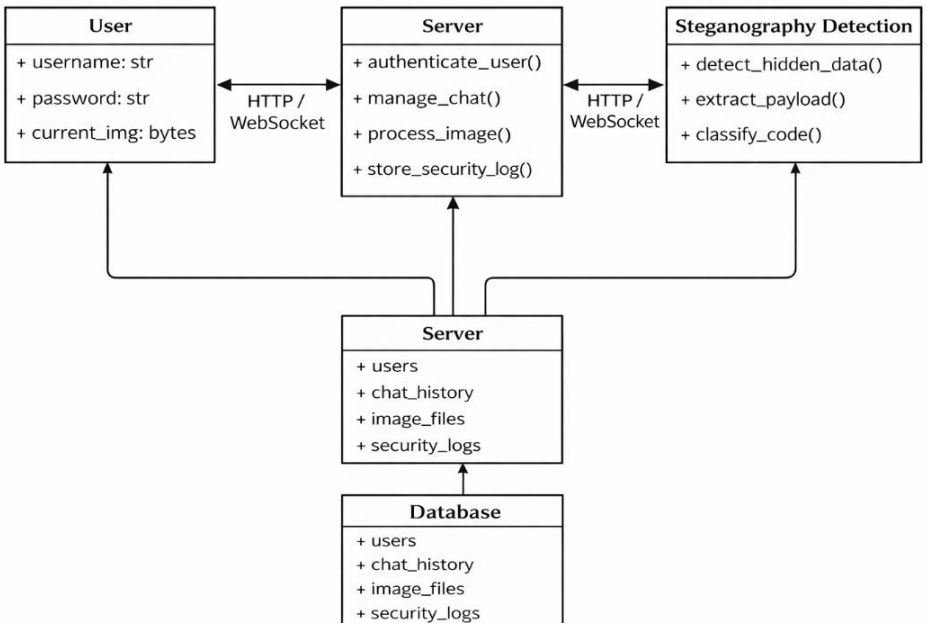

Fig: 4.3.4 CLASS DIAGRAM

4.3.5 SEQUENCE DIAGRAM

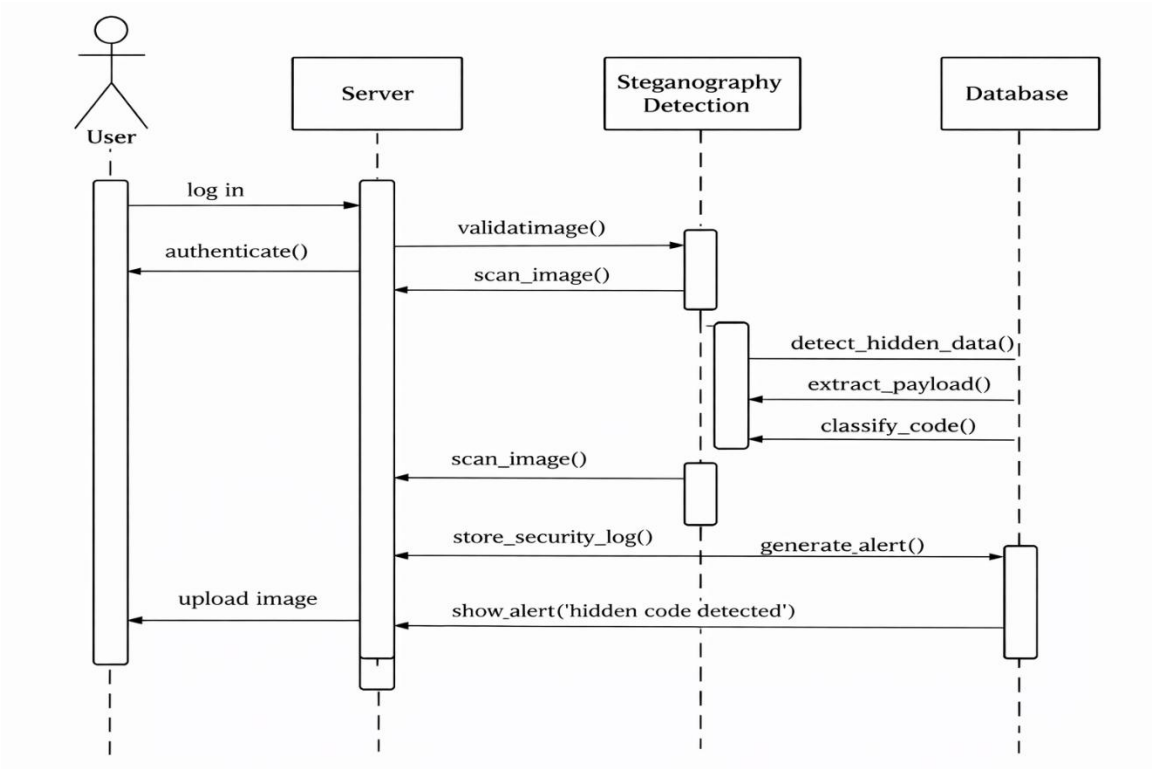

Fig: 4.3.5 SEQUENCE DIAGRAM

4.3.6 COMPONENT DIAGRAM

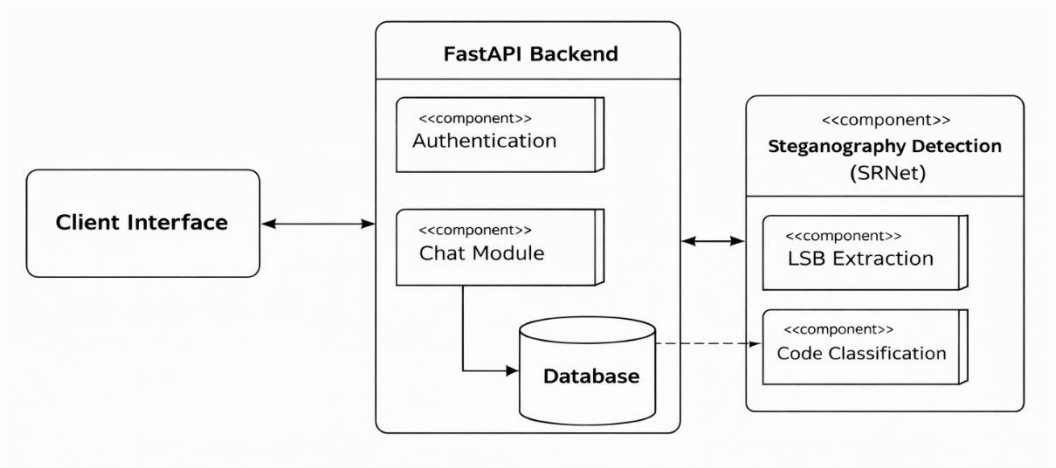

Fig 4.3.6 COMPONENT DIAGRAM

CHAPTER – 5

SYSTEM TESTING

Finding errors is the aim of testing. The process of testing a work product involves attempting to find every potential flaw or vulnerability.

5.1 TYPES OF TESTING

Software testing is again classified into two types that is manual testing and automation testing.

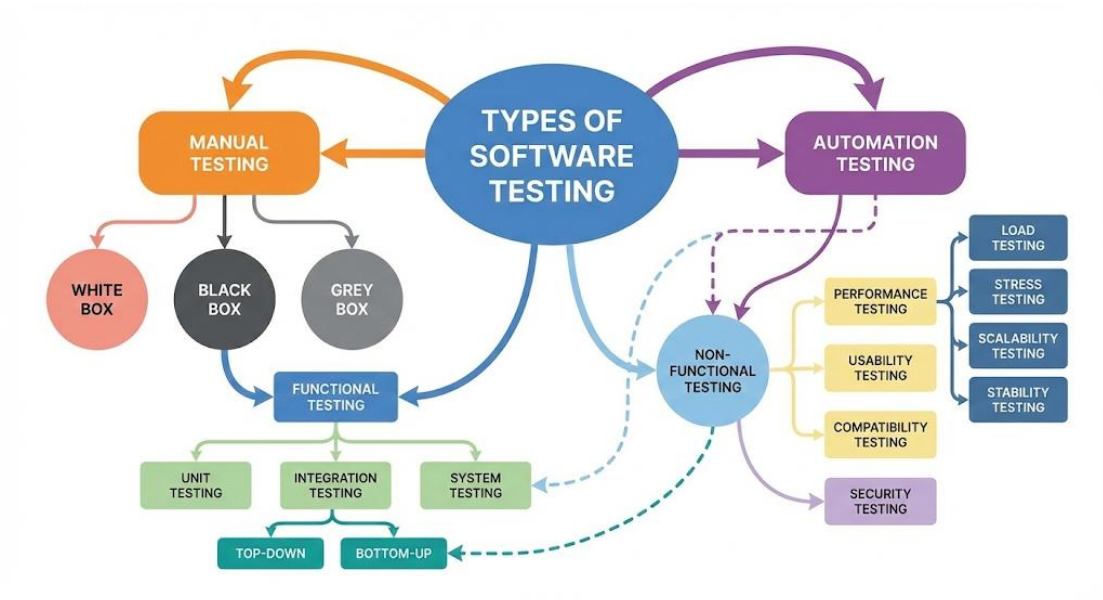

Fig:5.1.1 TYPES OF SOFTWARE TESTING

5.1.1 UNIT TESTING

Unit testing refers to dividing the entire Chat Application into smaller functional components such as authentication, message handling, image validation, steganography detection, payload extraction, and code classification, and testing each module independently to ensure correct functionality. Each unit is tested with predefined inputs to verify that it produces expected and valid outputs.

Unit testing validates the internal logic of the program, ensuring that individual functions perform according to design specifications. It checks decision-making blocks, input validations, and data processing operations within the system. This type of testing is structural in nature and relies on understanding the internal implementation of each module.

In this project, unit testing ensures that every stage of the steganography detection pipeline functions accurately and reliably. It verifies that image preprocessing works correctly, the SRNet detection model identifies hidden data properly, the LSB extraction

mechanism reconstructs payloads accurately, and the programming code classifier correctly identifies language patterns. Edge cases such as corrupted images, unsupported formats, or empty messages are also tested to ensure system robustness.

Unit testing is essential to guarantee that the system operates securely and efficiently during real-time communication.

1. Authentication Module

- Validate username and password inputs.
- Ensure secure login and session creation.
- Verify rejection of invalid credentials.

2. Image Validation and Preprocessing Module

- Check image format (PNG / JPEG).
- Verify file size constraints.
- Resize and normalize image before detection.
- Handle corrupted or incomplete image files.

3. Steganography Detection Module (SRNet)

- Test model loading and inference process.
- Verify correct classification as Stego or Clean.
- Validate output probability values.
- Ensure consistent prediction across test samples.

4. Hidden Payload Extraction Module

- Extract least significant bits correctly.
- Reconstruct binary stream into readable text.
- Handle cases where no hidden data exists.

5. Programming Code Classification Module

- Detect keywords and syntax patterns of programming languages.
- Verify correct identification of Python, C/C++, Java, HTML, SQL, etc.
- Ensure false positives are minimized.

Unit testing ensures that each independent component of the Chat Application performs correctly before integrating them into the complete system. This structured testing approach helps in building a reliable, secure, and scalable steganography detection system.

5.1.2 INTEGRATION TESTING

While unit testing verifies individual modules independently, integration testing ensures that all modules of the Chat Application work together as a complete and seamless

pipeline. In this project, integration testing validates the end-to-end flow from user input (text or image) to final detection result and alert generation. It checks how authentication, image validation, steganography detection, payload extraction, code classification, and logging interact in real-world scenarios.

In our project, the main objectives of integration testing are:

- The image flows from upload to final detection result without interruption.
- The detection module correctly triggers the extraction module when hidden data is identified.
- The extracted payload is properly passed to the code classification module.
- The system correctly generates alerts and logs events when suspicious content is detected.

Our hybrid security pipeline operates as follows:

- User Authentication
- Image Upload and Validation
- SRNet-based Steganography Detection
- LSB-based Hidden Payload Extraction
- Payload Reconstruction
- Programming Code Classification
- Security Alert and Logging
- Final Output Display

1. End-to-End Flow Testing

Test that an uploaded image passes through validation, detection, extraction, classification, and alert generation successfully, and that the final result is displayed correctly to the user.

2. Input/Output Compatibility Between Modules

Ensure that the output of the detection module is compatible with the input of the extraction module. Similarly, confirm that reconstructed payload text is properly formatted for the classification stage

3. Deep Learning Model Integration

Verify that the SRNet model loads correctly within the backend framework, performs inference without errors, and handles real-world image inputs reliably. The backend (FastAPI) ensures stable integration between AI modules and chat functionalities.

4. Error Handling and Fallback Mechanisms

Check system behavior with corrupted images, unsupported formats, empty payloads, or model failures. The system must handle exceptions gracefully and prevent crashes during real-time communication.

Tools Used for Integration Testing

- Pytest for integration test cases
- Mock testing to simulate detection and extraction stages
- PyTorch/TensorFlow model loading with error handling
- Sample image dataset for consistent integration validation

Integration testing plays a crucial role in validating the complete security pipeline of the Chat Application. Unlike unit testing, which focuses on isolated components, integration testing ensures smooth interaction between communication modules and AI-based detection mechanisms. Since the system operates in real time, proper integration guarantees reliability, security, and robustness against steganography-based threats.

5.1.3 FUNCTIONAL TESTING

Functional testing is an essential part of validating the performance and correctness of the Chat Application. It ensures that each functional requirement of the system operates as specified in the system design and technical documentation. In this project, functional testing verifies both communication features and the AI-based steganography detection pipeline.

Functional testing focuses on validating the system's ability to perform its intended tasks, including user authentication, real-time messaging, image upload, hidden data detection, payload extraction, programming code classification, and security alert generation. The testing ensures that each module produces accurate outputs for given inputs under normal operating conditions.

The detection functionality is tested to confirm that the SRNet model accurately identifies images containing hidden data, even when images vary in resolution, compression level, or minor distortions. The extraction process is validated to ensure that embedded payloads are correctly reconstructed without data corruption. The classification stage is functionally tested to verify that programming languages such as Python, C/C++, Java, HTML, or SQL are correctly identified when present.

Functional testing also ensures smooth interaction between all stages of the pipeline — image validation, detection, extraction, classification, and alert generation — so that each component correctly accepts inputs and produces expected outputs.

1. Steganography Detection Testing

- Evaluate how accurately the model detects hidden data in various image formats.
- Test detection performance on clean and stego images.
- Compare model predictions with known ground truth labels.

2. Payload Extraction Testing

- Verify that LSB extraction correctly retrieves embedded binary data.
- Test reconstruction of hidden text from extracted bits.
- Validate behavior when no hidden data is present.

3. Code Classification Testing

- Assess whether the system correctly identifies programming languages from extracted payloads.
- Measure accuracy, precision, recall, and F1-score for classification performance.
- Validate detection of suspicious keywords and executable patterns.

4. End-to-End Functional Validation

- Test full pipeline from image upload to final alert generation.
- Use known stego and non-stego images for validation.
- Check system behavior under edge cases such as corrupted images or unsupported formats.
- Verify proper alert notification and logging mechanisms.

Functional testing ensures that the Chat Application meets its functional requirements and operates reliably in real-time communication environments while maintaining high detection accuracy and system stability.

5.1.4 SYSTEM TESTING

System testing is performed to validate the complete Chat Application as an integrated and fully functional system. It ensures that all modules — including authentication, real-time messaging, image handling, steganography detection, payload extraction, classification, alert generation, and logging — operate together correctly in an end-to-end environment.

The primary objective of system testing is to verify the robustness, reliability, and overall functionality of the entire security pipeline. It confirms that when a user uploads an image, the system successfully processes it through validation, deep learning-based

detection (SRNet), LSB payload extraction, programming code classification, and alert generation without failure.

Objectives of System Testing

• Ensure the entire communication and detection pipeline works correctly from login to final output.

• Validate that the deep learning detection model and extraction algorithms are properly integrated.

• Confirm that suspicious content triggers appropriate alerts and logging mechanisms.

• Verify system stability under real-time messaging conditions.

Components Tested

• User authentication and session management.

• Real-time text message transmission.

• Image upload and format validation.

• Steganography detection accuracy using test image datasets.

• Payload extraction and reconstruction reliability.

• Programming code classification correctness.

• Security alert generation and log storage.

Tools and Technologies Used

• Python for backend implementation and system logic.

• FastAPI for server-side integration and API management.

• PyTorch / TensorFlow for SRNet model loading and inference.

• OpenCV and Pillow for image preprocessing and handling.

• SQLite / PostgreSQL for database operations.

System testing ensures that the Chat Application performs securely and efficiently under real-world conditions, maintaining both communication functionality and advanced steganography-based threat detection.

5.1.5 WHITE BOX TESTING

- Whitebox testing involves various deep learning examination for the internal working of the model it also ensures that each component functions are indeed.
- Whitebox testing ensures logical control flow and output of each module are correct or not.

- Test edge cases like extreme lightning occlusions have preprocessing check in the white box testing and for the segmentation fee known images are verify if it is boundaries are currently marked or not.
- In white box testing it checks the feature extractions like vanishing gradients layer outputs and activation distributions it also reshaped the images and makes it identical inputs.

5.1.6 BLACK BOX TESTING

- Black box testing is a method of software testing where internal workings of application are not known to test.
- Black box texting focuses on evaluating the systems functions against the
- expected outputs based on various input scenarios without knowledge of internal code or model architecture.
- The rejected or error message of unclear images shows the particular error in this process.
- It responds for time for different input sizes and it ensures that the system behaviour is under the batch of the input given by the user.
- It checks the test image sizes boundary limitations and checks the validation sets it also ensures the edge case scenarios.

5.2 TEST CASES

Test Case Table

S.No	Test Case ID	Test Case Description	Input	Expected Output	Actual Output
1	TC01	User Login Validation	Valid username & password	User successfully logged in	Login Successful
2	TC02	Invalid Login Check	Incorrect credentials	Access denied with error message	Error Message Displayed
3	TC03	Send Text Message	Text message input	Message delivered to receiver	Message Delivered

4	TC04	Image Upload Validation	PNG/JPEG image file	Image accepted and processed	Image Processed
5	TC05	Steganography Detection (Clean Image)	Clean image	No hidden data detected	Clean
6	TC06	Steganography Detection (Stego Image)	Image with hidden data	Hidden data detected	Suspicious
7	TC07	Payload Extraction	Stego image	Hidden text extracted successfully	Text Extracted
8	TC08	Programming Code Classification	Extracted payload text	Programming language identified	Language Detected (e.g., Python)
9	TC09	Security Alert Generation	Suspicious image with code	Alert notification generated	Alert Displayed

CHAPTER 6
SYSTEM IMPLEMENTATION

The implementation phase converts the designed architecture of the Chat Application into a fully functional system. This phase explains how different modules—such as user authentication, real-time messaging, image handling, steganography detection (SRNet), LSB payload extraction, programming code classification, and alert generation—are developed and deployed using Python-based technologies, FastAPI backend services, deep learning frameworks, and web deployment platforms.

DATASET DESCRIPTION – BOSSBase Dataset

The BOSSBase (Break Our Steganographic System Base) dataset is a widely used benchmark dataset for evaluating steganography and steganalysis algorithms. It was introduced to provide standardized image data for fair comparison of different steganographic detection methods.

BOSSBase 1.01 consists of 10,000 grayscale images with a resolution of 512 × 512 pixels, captured from seven different digital cameras. The images are uncompressed and contain diverse natural scenes, making them suitable for realistic steganographic embedding experiments. Researchers commonly generate stego images from this dataset using spatial-domain embedding techniques such as LSB, HUGO, WOW, and S-UNIWARD.

In this project, the BOSSBase dataset is used to train and evaluate the SRNet-based steganography detection model by creating both clean (cover) and stego image samples. Its standardized structure and high-quality images make it an ideal dataset for developing robust deep learning-based steganalysis systems.

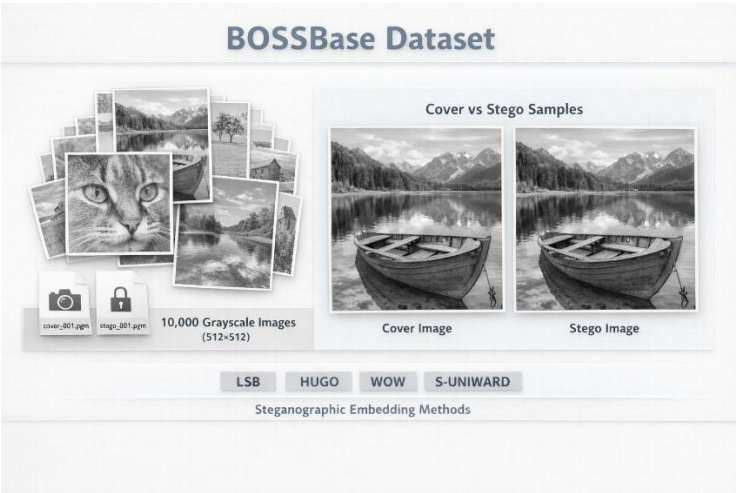

Fig 6.1 Dataset Description

Setting Up the Development Environment

Step 1: Install Python

1. Visit the official Python website.
2. Download Python 3.10 or above (64-bit recommended).
3. During installation, select "Add Python to PATH."
4. Complete the installation process.

Step 2: Create Virtual Environment

To isolate project dependencies:

```
python -m venv stegochat-env
```

Activate the environment:

Windows:

```
stegochat-env\Scripts\activate
```

macOS/Linux:

```
source stegochat-env/bin/activate
```

Step 3: Install Required Packages

Install necessary libraries for backend, AI model, and image processing:

```
pip install fastapi uvicorn
pip install torch torchvision
pip install opencv-python pillow numpy
pip install sqlalchemy
pip install python-multipart
```

Step 4: Backend Development (FastAPI)

- Create API endpoints for user registration and login.
- Implement message sending and receiving endpoints.
- Develop image upload API.
- Integrate SRNet model for steganography detection.
- Implement LSB extraction logic.
- Add code classification module.
- Implement security logging system.

Run the backend server:

```
uvicorn main:app --reload
```

Step 5: Model Integration

The SRNet deep learning model is loaded using PyTorch/TensorFlow.

• Load trained model weights.

• Perform inference on uploaded images.

• Return probability score for hidden data detection.

• Trigger extraction module if suspicious.

Step 6: Frontend Interface

• Develop login and registration pages.

• Implement chat interface with real-time messaging.

• Add image upload functionality.

• Display detection results and alerts dynamically.

• Provide security log viewing option.

Frontend technologies used:

- HTML
- CSS
- JavaScript

Step 7: Deployment

Backend Deployment:

- Deploy FastAPI backend on Render.
- Frontend Deployment:
- Deploy frontend on Vercel.
- Configure API base URL for production.
- Test communication between deployed frontend and backend.

Step 8: Running the Application

1. Start backend server (local or deployed).
2. Open frontend in browser.
3. Register or login.
4. Send text messages or upload images.
5. System automatically performs detection and displays results.

Image Upload and Preprocessing Module

The system begins with an interactive web-based user interface that allows authenticated users to send text messages and upload images in PNG or JPEG format. Once an image is uploaded through the chat interface, the backend automatically triggers the image validation and preprocessing module.

The preprocessing stage verifies file type and size, resizes the image to the required dimensions for the SRNet model, and normalizes pixel values to ensure consistency. Basic image integrity checks are performed to handle corrupted or unsupported files. These preprocessing steps prepare the image for accurate steganography detection while maintaining the integrity of pixel-level details necessary for hidden data analysis.

Model Loading and Detection Module

After preprocessing, the steganography detection model (SRNet) is loaded in the backend using deep learning frameworks such as PyTorch or TensorFlow. The model has been pre-trained to detect subtle pixel-level modifications introduced by steganographic embedding techniques such as LSB substitution.

The system automatically performs inference on the uploaded image without requiring manual model selection. The detection module outputs a probability score indicating whether the image contains hidden data. If the probability exceeds a predefined threshold, the system classifies the image as suspicious and activates the extraction module.

Payload Extraction and Code Classification

When hidden data is detected, the LSB extraction module retrieves the embedded binary information from the image. The binary stream is reconstructed into readable text format through payload reconstruction techniques.

The extracted text is then passed to the programming code classification module. This module analyzes syntactic patterns and keywords to determine whether the hidden content contains executable code such as Python, C/C++, Java, JavaScript, HTML, or SQL. A classification result along with confidence indication is generated.

The final output displayed to the user includes:

- Clean (No hidden data detected), or
- Suspicious (Hidden data detected),
- Programming language identified (if applicable),
- Confidence score,
- Security alert notification.

Deployment and Access

During development and testing, the system is hosted on a local server using FastAPI and Uvicorn. For production deployment, the backend is deployed on cloud platforms such as Render, while the frontend is hosted on platforms such as Vercel. The backend APIs manage communication between users and handle model inference, extraction logic, and database operations.

The system supports real-time communication while seamlessly integrating AI-based steganography detection. Deployment ensures scalability, secure API handling, and accessibility across devices with internet connectivity.

Software Installation for Chat Application with Steganography Detection

1. System Requirements

- **Operating System:** Windows 10/11, Ubuntu 18.04/20.04, or macOS
- **Python:** 3.9 or above (3.10 recommended)
- **RAM:** Minimum 8 GB (16 GB recommended for model training)
- **Storage:** At least 5–10 GB free space
- **GPU (Optional):** NVIDIA GPU with CUDA support for faster model inference

2. Install Python & Create Virtual Environment

Install Python from the official website and ensure it is added to PATH.

Create and activate a virtual environment:

```
# Install virtualenv (optional)
pip install virtualenv

# Create virtual environment
virtualenv stego-env

# Activate environment
source stego-env/bin/activate      # Linux/macOS
stego-env\Scripts\activate         # Windows
```

3. Install Required Libraries

Create a requirements.txt file with the following:

```
fastapi
uvicorn
torch
torchvision
opencv-python
pillow
numpy
scikit-learn
sqlalchemy
python-multipart
pytest
```

Then install dependencies:

```
pip install -r requirements.txt
```

4. (Optional) Install CUDA and cuDNN for GPU Support

If using an NVIDIA GPU:

• Install CUDA Toolkit

• Install cuDNN

• Verify installation:

python -c "import torch; print(torch.cuda.is_available())"

5. Project Structure Setup

Create the following folder structure:

```
stego-chat-application/
│
├── backend/
│   ├── main.py
│   ├── models/
│   ├── detection/
│   ├── extraction/
│   ├── classification/
│   └── database/
│
├── frontend/
│   ├── index.html
│   ├── chat.html
│   ├── style.css
│   └── script.js
│
├── requirements.txt
└── README.md
```

6. Run the Backend Server

- Navigate to backend directory and run:
- uvicorn main:app --reload
- The server will start at:
- http://127.0.0.1:8000

7. Model Integration

• Place trained SRNet model inside /models directory.

• Ensure model weights are correctly loaded in the detection module.

• Verify model inference using test images before deployment.

8. Deployment Setup

Backend Deployment:

• Deploy FastAPI backend to Render or similar cloud service.

Frontend Deployment:

• Deploy frontend to Vercel.

• Update API base URL in frontend configuration.

• Test API connectivity after deployment.

Image Processing Libraries

- **OpenCV:** Used for image validation and LSB extraction.
- **Pillow (PIL):** Used for image loading and format handling.
- **NumPy:** Used for pixel-level data manipulation.

Preprocessing and Data Handling

- Resize image to model-compatible dimensions.
- Normalize pixel values.

Evaluation Metrics & Formulas

Let:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

1. Accuracy

Percentage of total correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision

How many of the predicted positives were actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. Recall (Sensitivity)

How many actual positives were correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. F1-Score

Harmonic mean of Precision and Recall.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION

This project has explored the integration of deep learning and traditional algorithms for enhancing cybersecurity within a real-time Chat Application. The system demonstrates how a hybrid steganography detection framework—combining SRNet-based deep learning detection with LSB payload extraction and programming code classification—can effectively identify hidden data embedded within shared images.

Through systematic design and implementation, the application successfully enables secure communication while automatically scanning and analyzing multimedia content. The deep learning model detects subtle pixel-level modifications, while the extraction and classification modules reconstruct hidden payloads and determine whether executable programming code is present. This multi-layered approach significantly strengthens content-level security beyond conventional encryption-based protection.

The system reduces the risk of covert data transmission and hidden code injection attacks by generating automated alerts and maintaining structured security logs. Its modular architecture ensures scalability, allowing future improvements such as advanced AI-based threat intelligence, improved model accuracy, and multi-user expansion.

Although challenges such as model generalization, computational efficiency, and evolving steganographic techniques remain, the project establishes a strong foundation for intelligent, AI-driven secure communication systems. Overall, the developed Chat Application demonstrates the practical feasibility of integrating artificial intelligence into cybersecurity frameworks to proactively detect and mitigate hidden digital threats.

FUTURE ENHANCEMENTS

The developed Chat Application with hybrid steganography detection provides a strong foundation for secure multimedia communication; however, several enhancements can be implemented in the future to improve performance, scalability, and intelligence.

Future improvements may include the integration of more advanced deep learning architectures such as Vision Transformers or ensemble-based steganalysis models to enhance detection accuracy and robustness against complex embedding techniques. The system can also be extended to detect hidden data in other media formats such as audio, video, and document files, thereby broadening its cybersecurity capabilities.

Another important enhancement is the implementation of real-time adversarial attack detection to identify intentionally manipulated images designed to evade steganalysis

models. Continuous model retraining using updated datasets can improve generalization and adaptability to evolving steganographic methods.

The application can also be expanded into a fully scalable multi-user secure communication platform with end-to-end encryption combined with AI-driven content inspection. Integration with cloud-based threat intelligence systems and automated incident response mechanisms can further strengthen cybersecurity resilience.

Overall, the project opens opportunities for developing intelligent, AI-powered communication systems capable of proactively detecting and preventing covert digital threats in modern network environments.

FUTURE SCOPE

1. **Multi-Class Hidden Content Classification:** Extend the system to classify different types of hidden payloads such as scripts, malware signatures, encrypted data, or confidential text.
2. **Larger and Diverse Datasets:** Train the detection model using larger and more diverse steganographic datasets to improve robustness and generalization.
3. **Real-Time Optimized Detection:** Optimize model inference speed for instant detection in high-traffic, real-time communication environments.
4. **Cross-Media Steganography Detection:** Expand detection capabilities to audio, video, and document-based steganography.
5. **Advanced User Interface Enhancements:** Develop a more interactive web/mobile interface with real-time threat visualization and detailed security analytics.
6. **Explainable AI Integration:** Incorporate explainability tools to highlight image regions responsible for hidden data detection.
7. **Privacy-Preserving Learning:** Implement federated learning techniques to improve model performance without sharing sensitive data.
8. **Enterprise-Level Deployment:** Integrate the system with enterprise security infrastructures and cloud-based threat monitoring platforms.

REFERENCES

[1] A. Almhlbdi et al., “Deep Learning-Based Multi-Class Detection of LSB Steganography in Digital Images,” *IEEE Access*, vol. 13, pp. XXXX–XXXX, Nov. 2025.

[2] S. Li, Y. Zhang, and H. Wang, “Image Steganalysis Using Active Learning and Hyperparameter Optimization,” *IEEE Trans. Information Forensics and Security*, 2025.

[3] M. Alrusaini, “Robust Deep Learning-Based Steganalysis Under Image Transformations,” *IEEE Access*, 2025.

[4] R. Mostafa, K. El-Fishawy, and O. M. Doss, “Deep Learning-Based Detection and Classification of Hidden Data in Digital Images,” *IEEE Access*, 2024.

[5] E. Kuchumova et al., “STEG-XAI: Explainable Artificial Intelligence for Image Steganalysis,” *IEEE Access*, 2023.

[6] J. Ntivuguruzwa and T. Ahmad, “A Convolutional Neural Network Architecture for Spatial Domain Steganalysis,” *Multimedia Tools and Applications*, 2023.

[7] M. Boroumand, M. Chen, and J. Fridrich, “Deep Residual Network for Steganalysis of Digital Images,” *IEEE Trans. Information Forensics and Security*, vol. 14, no. 5, pp. 1181–1193, 2019.

[8] Y. Yedroudj, F. Comby, and M. Chaumont, “Yedroudj-Net: An Efficient CNN for Spatial Steganalysis,” *IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2018.

[9] B. Li, M. Wang, J. Huang, and X. Li, “A New Cost Function for Spatial Image Steganography,” *IEEE Trans. Image Processing*, vol. 23, no. 9, pp. 4088–4100, 2014.

[10] J. Fridrich and J. Kodovský, “Rich Models for Steganalysis of Digital Images,” *IEEE Trans. Information Forensics and Security*, vol. 7, no. 3, pp. 868–882, 2012.

[11] V. Holub and J. Fridrich, “Designing Steganographic Distortion Using Directional Filters,” *IEEE Int. Workshop on Information Forensics and Security (WIFS)*, 2012.

[12] T. Pevný, P. Bas, and J. Fridrich, “Steganalysis by Subtractive Pixel Adjacency Matrix,” *IEEE Trans. Information Forensics and Security*, vol. 5, no. 2, pp. 215–224, 2010.

[13] I. Goodfellow et al., “Generative Adversarial Nets,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[14] L. Niu, W. Zhang, X. Yang, and N. Yu, “GAN-Based Steganography and Steganalysis,” *IEEE Signal Processing Magazine*, vol. 35, no. 5, pp. 44–54, 2018.

[15] P. Bas, T. Filler, and T. Pevný, “Break Our Steganographic System: The Ins and Outs of Organizing BOSS,” *Information Hiding Conference*, 2011.

APPENDIX

A. SOURCE CODE

FRONT-END CODE

chat.html:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Secure Stego Chat</title>
  <link rel="stylesheet" href="style.css">
</head>
<body data-page="chat">
  <header class="topbar">
    <div class="topbar-brand">
      <h1>🛡 StegoShield AI</h1>
      <span class="topbar-subtitle">AI-Powered Secure Messaging</span>
    </div>
    <div class="topbar-actions">
      <span id="me-name"></span>
      <button id="security-logs-btn" class="btn-sm btn-outline" title="Security Logs">
        <svg viewBox="0 0 24 24" width="14" height="14" fill="currentColor"
          style="vertical-align:-2px;margin-right:4px">
          <path
            d="M12 1L3 5v6c0 5.55 3.84 10.74 9 12 5.16-1.26 9-6.45 9-12V5l-9-4zm0 10.99h7c-.53 4.12-3.28
7.79-7 8.94V12H5V6.3l7-3.11v8.8z" />
        </svg>
        Security Logs
      </button>
      <button id="logout" class="btn-sm btn-danger-outline">Logout</button>
    </div>
  </header>
  <main class="chat-shell">
    <aside class="users-panel">
      <div class="sidebar-header">
        <h2>Contacts</h2>
        <input id="user-search" type="text" placeholder="Search users…" autocomplete="off">
      </div>
      <ul id="users-list"></ul>
      <p id="no-users" style="display:none">No registered users yet.<br>Ask someone to register!</p>
    </aside>
    <section class="chat-panel">
      <div class="chat-header">
        <div class="chat-header-avatar" id="chat-avatar"></div>
        <div>
          <div id="chat-title">Select a user to start chatting</div>
          <div id="chat-subtitle">Choose from contacts</div>
        </div>
      </div>
```

```
    <div id="messages" class="messages"></div>
    <div class="composer-wrap">
      <form id="text-form" class="composer">
        <input id="text-input" type="text" placeholder="Type a message…" autocomplete="off">
        <button type="submit" class="send-btn" title="Send">
          <svg viewBox="0 0 24 24" width="18" height="18" fill="currentColor">
            <path d="M2.01 21L23 12 2.01 3 2 10l15 2-15 2z" />
          </svg>
        </button>
      </form>
      <form id="image-form" class="image-upload">
        <label class="file-label" for="image-input">
          <svg viewBox="0 0 24 24" width="14" height="14" fill="currentColor">
            <path
              d="M21 19V5c0-1.1-.9-2-2-2H5c-1.1 0-2 .9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2zM8.5 13.5l2.5
3.01L14.5 12l4.5 6H5l3.5-4.5z" />
          </svg>
          Attach Image
        </label>
        <input id="image-input" type="file" accept="image/*">
        <span class="file-name" id="file-name-display"></span>
        <button type="submit" class="send-img-btn">Send Image</button>
      </form>
      <p id="chat-error"></p>
    </div>
  </section>
 </main>
 <script src="script.js"></script>
</body>
</html>
```

Index.html

```
<!doctype html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>StegoShield — Secure Chat</title>
 <link rel="stylesheet" href="style.css">
</head>

<body data-page="auth">
 <main class="auth-shell">
  <section class="card">
   <h1>🛡 StegoShield AI</h1>
   <p class="muted">AI-Powered Secure Messaging</p>
   <div class="tabs">
    <button id="show-login" class="tab active">Login</button>
    <button id="show-register" class="tab">Register</button>
   </div>
```

```
    <form id="login-form" class="form active">
      <label>Username<input name="username" required></label>
      <label>Password<input name="password" type="password" required></label>
      <button type="submit">Sign In</button>
    </form>

    <form id="register-form" class="form">
      <label>Username<input name="username" required></label>
      <label>Password<input name="password" type="password" required></label>
      <button type="submit">Create Account</button>
    </form>
    <p id="auth-error" class="error"></p>
  </section>
 </main>
 <script src="script.js"></script>
</body>

</html>
```

BACK-END CODE:

Training Code:

```
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader, random_split
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
from PIL import Image
from tqdm import tqdm
COVER_PATH = r"J:\project\backend\dataset\BOSSbase_1.01\covers"
STEGO_PATH = r"J:\project\backend\dataset\BOSSbase_1.01\stego"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
BATCH_SIZE = 4      # 512 images require smaller batch
EPOCHS = 6
LR = 5e-4
class StegoDataset(Dataset):
    def __init__(self, cover_dir, stego_dir):
        self.samples = []
        for img in os.listdir(cover_dir):
            if img.endswith(".pgm"):
                self.samples.append((os.path.join(cover_dir, img), 0))
        for img in os.listdir(stego_dir):
            if img.endswith(".pgm"):
                self.samples.append((os.path.join(stego_dir, img), 1))
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("L")
        img = np.array(img, dtype=np.float32) / 255.0
        img = (img - 0.5) / 0.5  # zero center
```

```
        img = np.expand_dims(img, axis=0)
        return torch.tensor(img), torch.tensor(label)
dataset = StegoDataset(COVER_PATH, STEGO_PATH)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
print("Train samples:", len(train_dataset))
print("Validation samples:", len(val_dataset))
class StegoCNNWithHPF(nn.Module):
    def __init__(self):
        super(StegoCNNWithHPF, self).__init__()
        # High-pass filter (fixed)
        self.hpf = nn.Conv2d(1, 1, kernel_size=3, padding=1, bias=False)
        # Laplacian kernel
        hpf_kernel = torch.tensor([[
            [0, -1, 0],
            [-1, 4, -1],
            [0, -1, 0]
        ]], dtype=torch.float32).unsqueeze(0)
        self.hpf.weight = nn.Parameter(hpf_kernel, requires_grad=False)
        self.conv = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 32 * 32, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 2)
        )
    def forward(self, x):
        x = self.hpf(x)
        x = self.conv(x)
        x = self.fc(x)
        return x
model = StegoCNNWithHPF().to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)
train_losses = []
val_losses = []
```

```
train_accs = []
val_accs = []
for epoch in range(EPOCHS):
    model.train()
    running_loss = 0
    correct = 0
    total = 0
    for images, labels in tqdm(train_loader):
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)
    train_loss = running_loss / len(train_loader)
    train_acc = correct / total
    train_losses.append(train_loss)
    train_accs.append(train_acc)
    model.eval()
    val_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)
    val_loss /= len(val_loader)
    val_acc = correct / total
    val_losses.append(val_loss)
    val_accs.append(val_acc)
    print(f"\nEpoch {epoch+1}/{EPOCHS}")
    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
    print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")
plt.figure(figsize=(8, 5))
plt.plot(epochs, train_acc, 'b-o', label='Training Accuracy')
plt.title('Training Accuracy vs Epoch')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.show()
```

detector.py

```
from PIL import Image
_DELIMITER = "#####"
def detect_lsb_steganography(image_path: str) -> tuple[bool, float]:
    """
    Two-pass LSB steganography detection:
    1. Definitive check — attempt LSB extraction and look for the delimiter.
    2. Statistical heuristic — chi-square test on the first portion of pixels
       where payloads are typically embedded.
    Returns (likely_hidden_data, confidence_percent).
    """
    try:
        with Image.open(image_path).convert("L") as img:
            pixels = list(img.getdata())
    except Exception:
        return False, 0.0
    # ── Pass 1: Try to find the delimiter in the LSB stream ──
    bits = "".join(str(p & 1) for p in pixels)
    extracted_chars = []
    for i in range(0, min(len(bits), 80_000), 8):  # scan first ~10 KB
        chunk = bits[i : i + 8]
        if len(chunk) < 8:
            break
        char = chr(int(chunk, 2))
        extracted_chars.append(char)
        if len(extracted_chars) >= 5:
            tail = "".join(extracted_chars[-5:])
            if tail == _DELIMITER:
                return True, 100.0
    sample_size = max(256, len(pixels) // 5)
    sample = pixels[:sample_size]

    # Pair adjacent even/odd values and test if LSBs are uniformly distributed.
    pairs = {}
    for p in sample:
        base = p & ~1  # strip LSB
        pairs.setdefault(base, [0, 0])
        pairs[base][p & 1] += 1
    chi_sq = 0.0
    n_pairs = 0
    for base, (c0, c1) in pairs.items():
        total = c0 + c1
        if total < 2:
            continue
        expected = total / 2
        chi_sq += ((c0 - expected) ** 2 + (c1 - expected) ** 2) / expected
        n_pairs += 1
    if n_pairs == 0:
        return False, 0.0
    # Normalise: perfectly uniform LSBs → high chi-sq per pair.
    avg_chi = chi_sq / n_pairs
    # avg_chi close to 0 means the distribution is very uniform (suspicious).
```

```
    # Natural images typically have avg_chi > 1.
    if avg_chi < 0.3:
        confidence = min(95.0, (0.3 - avg_chi) / 0.3 * 95)
        return True, round(confidence, 2)
    return False, round(max(0.0, (1.0 - avg_chi) * 50), 2)
```

extractor.py

```
from PIL import Image
import string
_DELIMITER = "#####"
_PRINTABLE = set(string.printable)
def extract_lsb_data(image_path: str) -> str:
    """
    Reads LSB stream and returns printable extracted text.
    Stops on delimiter or first non-printable byte.
    """
    try:
        with Image.open(image_path).convert("L") as img:
            pixels = list(img.getdata())
        bits = "".join(str(p & 1) for p in pixels)
        out = []
        for i in range(0, len(bits), 8):
            chunk = bits[i:i + 8]
            if len(chunk) < 8:
                break
            char = chr(int(chunk, 2))
            if char not in _PRINTABLE:
                break
            out.append(char)
            joined = "".join(out)
            if _DELIMITER in joined:
                return joined.replace(_DELIMITER, "").strip()
        return "".join(out).strip()
    except Exception:
        return ""
```

classifier.py

```
import re
_PATTERNS = {
    "python": [r"\bdef\s+\w+\(", r"\bimport\s+\w+", r"\breturn\b", r"\bclass\s+\w+"],
    "c_cpp": [r"#include\s*<", r"\bint\s+main\s*\(", r"std::", r"printf\s*\("],
    "java": [r"\bpublic\s+class\b", r"\bSystem\.out\.println\s*\(", r"\bstatic\s+void\s+main\b"],
    "html": [r"<html", r"<script", r"<div", r"<!DOCTYPE\s+html"],
    "javascript": [r"\bfunction\b", r"=>", r"console\.log\s*\(", r"\b(let|const|var)\s+\w+"],
                "sql": [r"\bSELECT\b", r"\bINSERT\s+INTO\b", r"\bUPDATE\s+\w+\s+SET\b",
r"\bDELETE\s+FROM\b"],
}

def classify_extracted_text(text: str) -> tuple[bool, str | None, int, list[str]]:
    if not text or len(text.strip()) < 12:
        return False, None, 0, []
    scores = {}
    matches = {}
    for language, patterns in _PATTERNS.items():
```

```
    found = []
    for pattern in patterns:
        if re.search(pattern, text, re.IGNORECASE | re.MULTILINE):
            found.append(pattern)
    scores[language] = len(found)
    matches[language] = found
top_language = max(scores, key=scores.get)
top_score = scores[top_language]
is_code = top_score >= 2
confidence = min(95, top_score * 25)
return is_code, (top_language if is_code else None), confidence, matches[top_language]
```

B. SCREENSHOTS

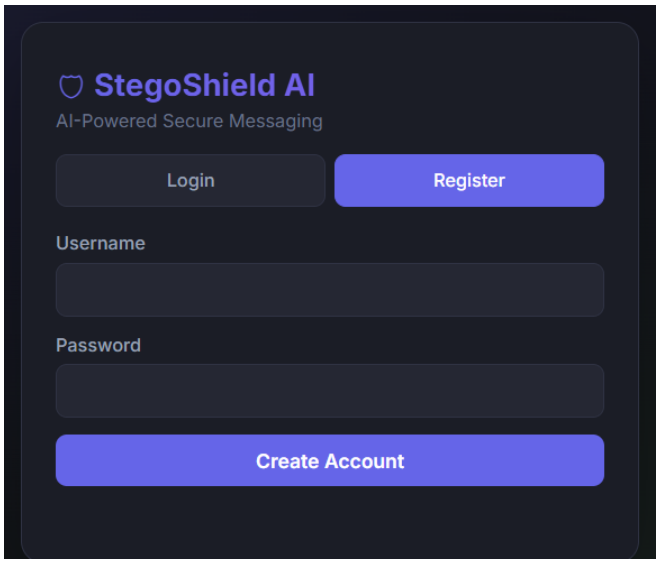

Fig B. a: Registration Page

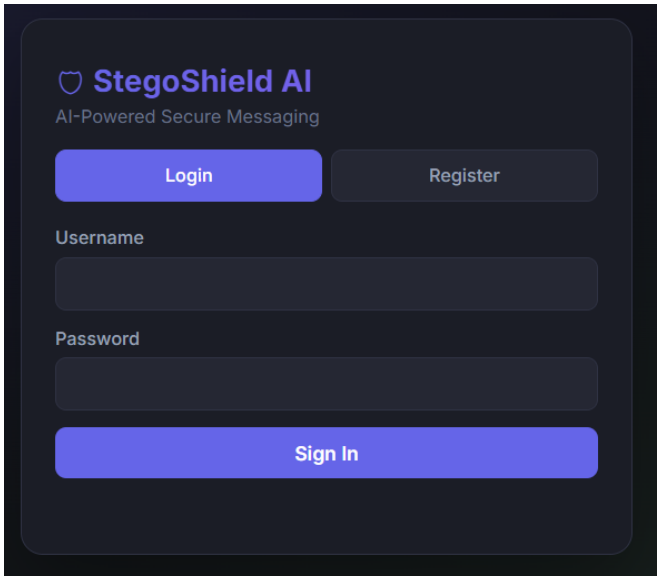

Fig B. b: Login Page

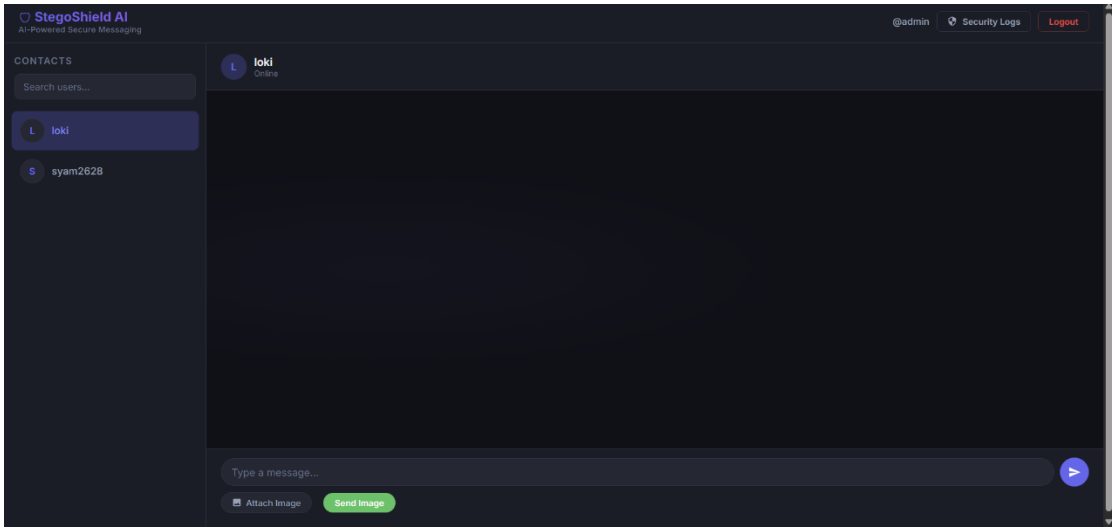

Fig B. c: Chat Interface

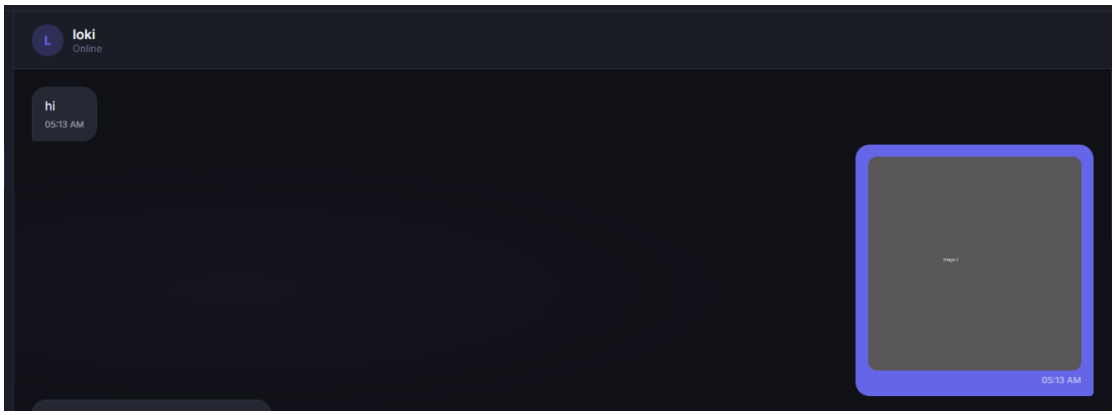

Fig B. d: Messaging

Fig B. e: Showing Suspicious for Stego Images

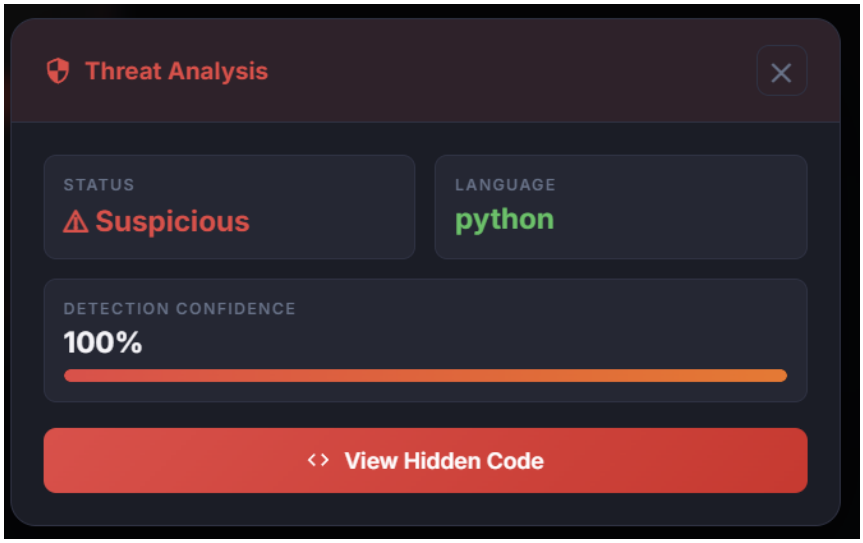

Fig B. f: Threat Analysis

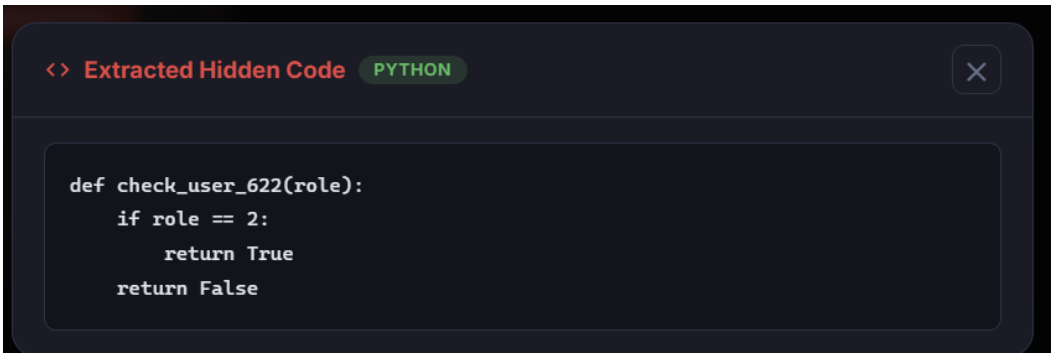

Fig B. g: Extracted Hidden code inside the Image

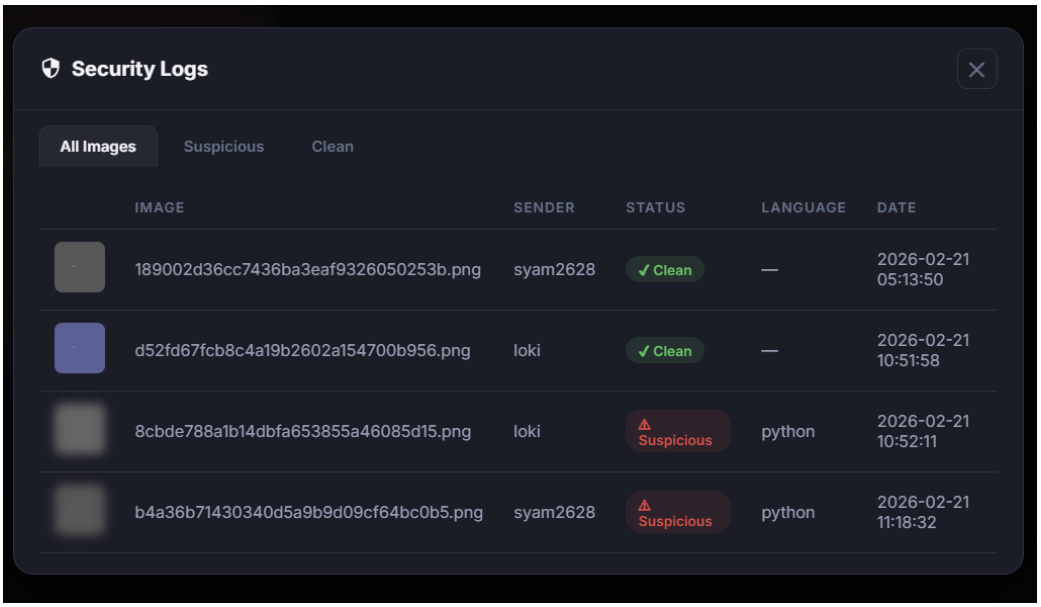

Fig B. h: Security Logs for Future Reference

C. GRAPHS

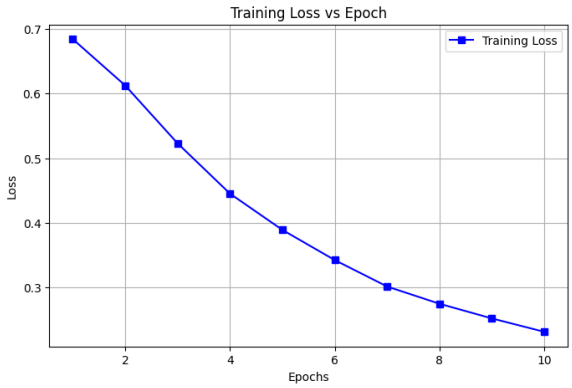

Fig C. a: Training Loss

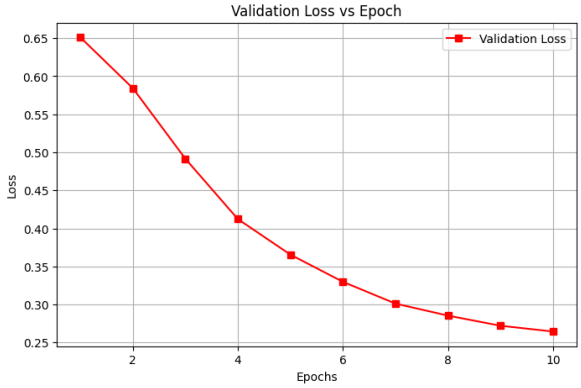

Fig C. b: Validation Loss

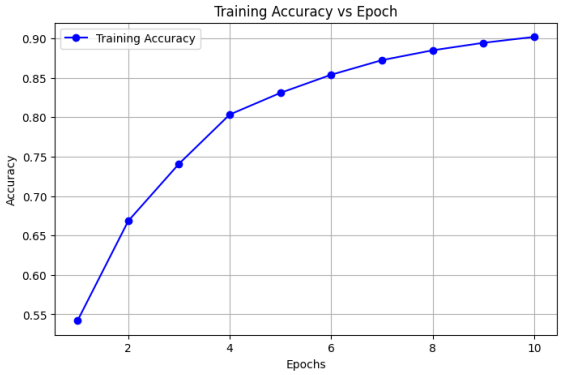

Fig C. c: Training Accuracy

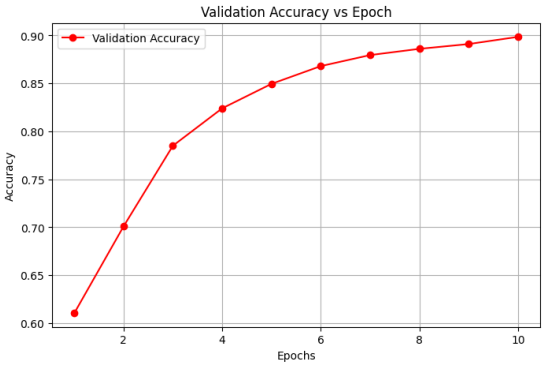

Fig C. d: Validation Accuracy

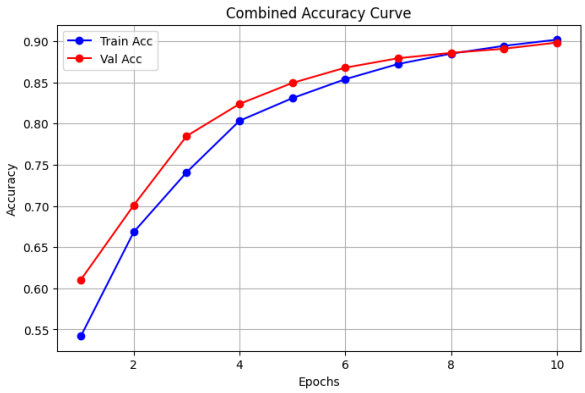

Fig C. e: Training vs Validation Accuracy

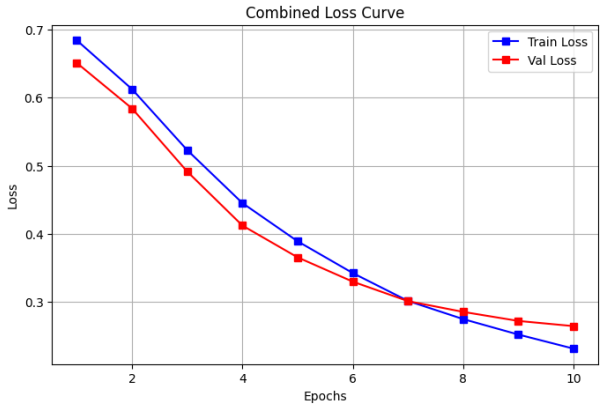

Fig C. f: Training and Validation Loss

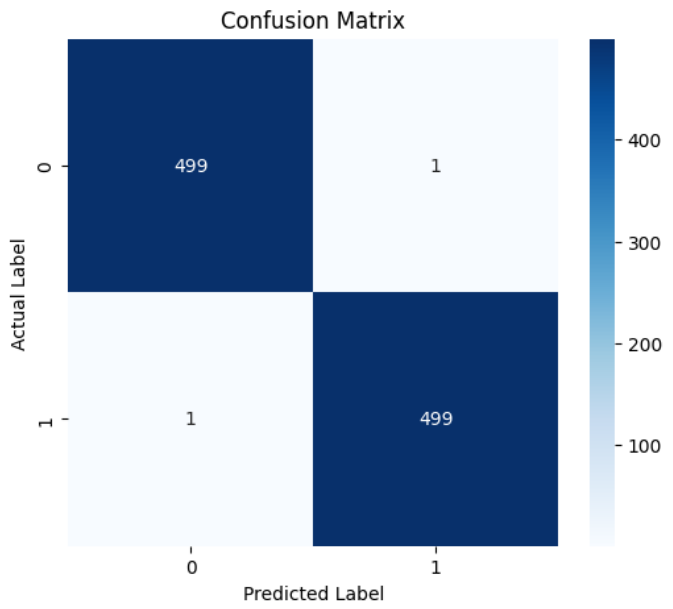

Fig C. g: Confusion Matrix

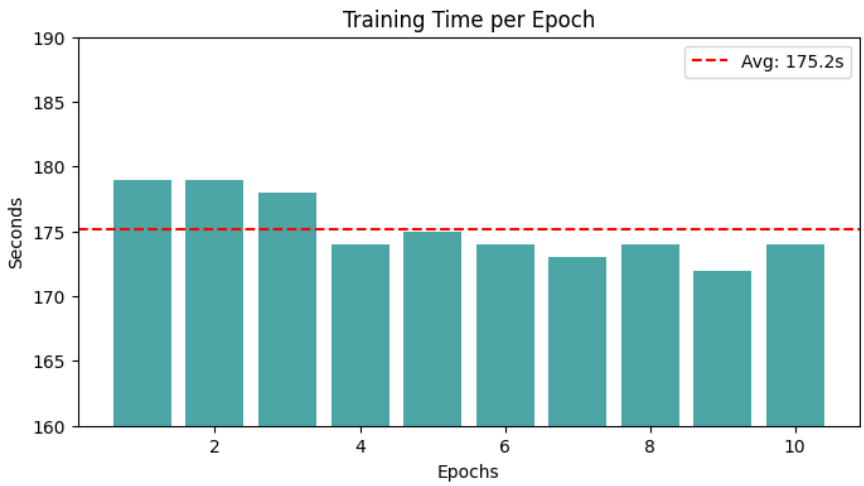

Fig C. h: Training Time vs Epochs

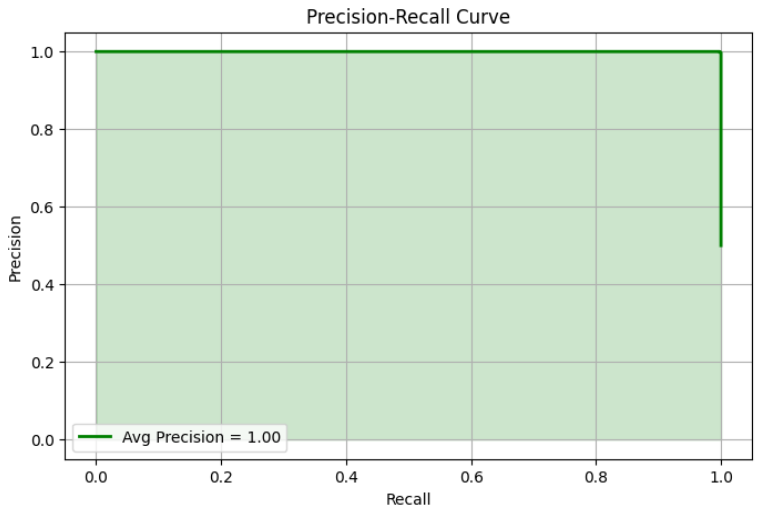

Fig C. i: Precision vs Recall Curve

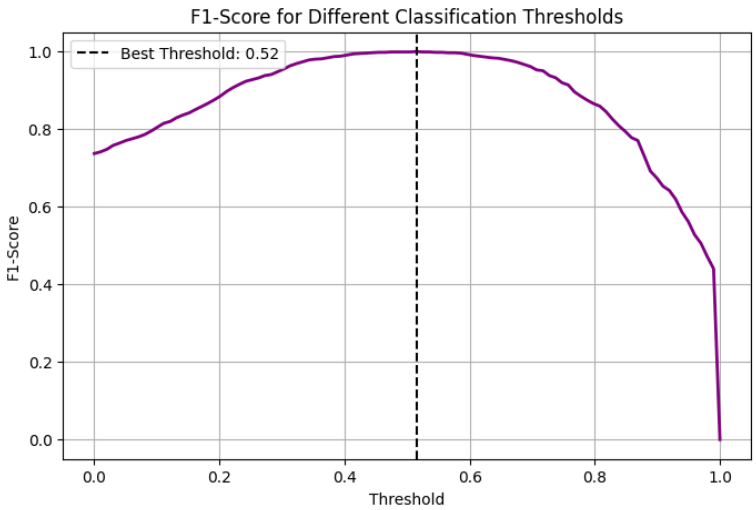

Fig C. j: F1- Score Curve

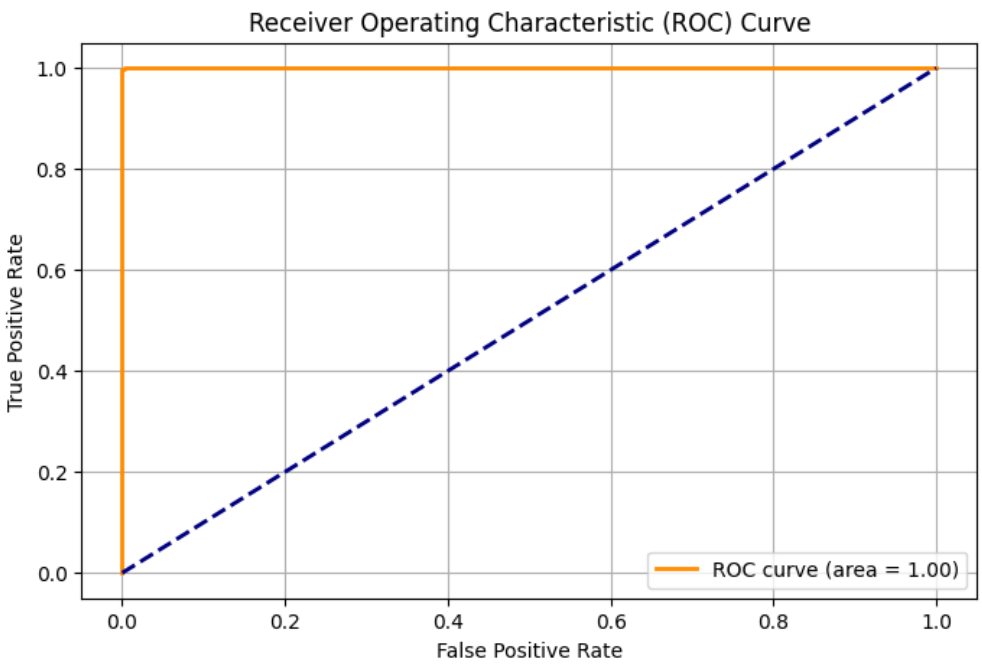

Fig C. k: ROC Curve

ANNEXURE I

Title of the Project	AI-Powered Secure Chat System with Automated Stegano Detection
Name of the Student	G V Syam Prasad – 23755A3101
	V Lokendra – 23755A3102
	Y Manoj Reddy – 22751A3172
	S Mokshith Reddy – 22751A3163
Name of the Guide & Designation	Ms. M. Deepika, Assistant Professor

Table 1: Outcomes attained and its Justification

PO	Justification
PO1	Gained basic knowledge on the project area while selecting the problem.
PO2	Learned how to analyze the complex problems.
PO3	Learned problem solving skills while implementing modules.
PO4	Learned how to conduct investigations on complex real-world problems.
PO5	Learned modern tool usage while developing source code.
PO6	Learned professional engineering practices while adding new features to the project.
PO7	Learned how this project impacts on societal and environmental contexts.
PO8	Learned software ethics to be followed while developing this project.
PO9	Learned how to work as an individual and as a team member during this entire project period.
PO10	Learned how to communicate effectively during the project reviews.
PO11	Learned how to estimate cost benefit analysis for the project management.
PO12	The knowledge which I have gained through the project will be helpful for my future to refine my skills.